

Uniwersytet Łódzki
Wydział Fizyki i Informatyki Stosowanej

Ochrona danych

Notatka do kolokwium

Kryptografia klasyczna, asymetryczna, funkcje hashujące
oraz podstawy arytmetyki modularnej

Imię i nazwisko: **Katarzyna Stańczyk**

Kierunek: **Informatyka stosowana**

Rok akademicki: **2025/2026**

Łódź, 26 maja 2026

Spis treści

1	Matematyka modularna i podstawy	3
1.1	Arytmetyka modularna	3
1.2	Szybkie potęgowanie modularne	11
1.3	Rozszerzony algorytm Euklidesa	18
1.4	Odwrotność modularna	24
1.5	NWD / gcd	30
1.6	Funkcja Eulera $\varphi(n)$	36
1.7	Test Division (metoda próbnego dzielenia)	41
1.8	Test pierwszościci liczb	47
1.9	Sito Eratostenesa	53
1.10	Test Lucasa–Lehmera	57
1.11	Liczby Mersenne’a	63
1.12	Generator modulo p	69
1.13	Logarytm dyskretny	74
2	Klasyczne szyfry i analiza	80
2.1	Szyfr Cezara	80
2.2	Szyfr Vigenère’a	86
2.3	Index of Coincidence (IC)	92
2.4	Metoda Kasiski	97
2.5	ASCII / UTF-8 / reprezentacja tekstu	102
3	Kryptografia asymetryczna	107
3.1	Diffie–Hellman	107
3.2	Man In The Middle Attack (MITM)	113
3.3	RSA	118
3.4	Zamiana tekstu na liczby / bloki bajtowe	125
3.5	ElGamal	130
3.6	Klucz publiczny vs klucz prywatny	137
3.7	Szyfrowanie asymetryczne vs symetryczne	141
3.8	Dlaczego RSA działa	146
3.9	Known Plaintext Attack (na przykładzie ElGamala)	152
4	OpenPGP i podpis cyfrowy	157
4.1	OpenPGP Studio	158
4.2	Podpis cyfrowy / Verify	162
4.3	Integralność danych	166
5	Hashe i kolizje	170
5.1	Funkcje hashujące / funkcje skrótu	171
5.2	DJB / DJB2	174
5.3	Adler32	179
5.4	Kolizje hashy	184
5.5	Birthday Paradox	188
5.6	Szansa kolizji i wzór birthday paradox	192
5.7	Rainbow Tables	197
5.8	Redukcja	201

5.9	Tablice hashów	205
5.10	Łańcuchy hash $\rightarrow$ redukcja $\rightarrow$ hash	209
5.11	Operacje bitowe ($\ll$, $\gg$, AND, maski bitowe)	213

1 Matematyka modularna i podstawy

1.1 Arytmetyka modularna

Definicja

Arytmetyka modularna polega na wykonywaniu działań na resztach z dzielenia przez liczbę n .

Używana jest praktycznie we wszystkich algorytmach kryptograficznych:

- RSA,
- Diffie–Hellman,
- ElGamal,
- funkcje hashujące.

Najważniejszy pomysł: po wykonaniu działania interesuje nas tylko reszta z dzielenia.

Najważniejsze wzory

$$a \bmod n = r$$

$$(a + b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$$

$$(a - b) \bmod n = ((a \bmod n) - (b \bmod n)) \bmod n$$

$$(a \cdot b) \bmod n = ((a \bmod n)(b \bmod n)) \bmod n$$

Najważniejsza własność z zajęć:

$$(A \cdot B) \bmod n = ((A \bmod n)(B \bmod n)) \bmod n$$

Własności i zasady

- Można redukować liczby modulo po każdym kroku.
- Dodawanie i mnożenie działa modulo.
- Duże liczby można zastępować ich resztami.
- Dzielenie modulo NIE działa normalnie.
- Przy odejmowaniu wynik może być ujemny:

$$-2 \bmod 7 = 5$$

Metoda krok po kroku

Jak liczyć działania modulo ręcznie:

1. Wykonuj działania etapami.
2. Po każdym kroku redukuj wynik modulo.
3. Nie licz ogromnych liczb bezpośrednio.
4. W potęgach używaj szybkiego potęgowania modularnego.

Schemat:

$$123^{456} \mod 789$$

1. Rozbij wykładnik.
2. Licz kolejne potęgi modulo.
3. Redukuj po każdym mnożeniu.

Przykład z zajęć

$$17 \mod 5 = 2$$

ponieważ:

$$17 = 3 \cdot 5 + 2$$

Przykład mnożenia:

$$(13 \cdot 17) \mod 5$$

Redukujemy:

$$13 \mod 5 = 3$$

$$17 \mod 5 = 2$$

Czyli:

$$(13 \cdot 17) \mod 5 = (3 \cdot 2) \mod 5 = 6 \mod 5 = 1$$

Typowe pułapki

- Nie wolno dzielić modulo bez odwrotności modularnej.
- Nie licz ogromnych potęg „normalnie”.
- Zawsze redukuj liczby modulo po drodze.
- Ujemne wyniki trzeba sprowadzić do dodatnich modulo.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Oblicz zwykłe modulo.

$$17 \mod 5$$

Rozwiązanie:

$$17 = 3 \cdot 5 + 2$$

$$17 \mod 5 = 2$$

Odpowiedź:

$$2$$

Zadanie 2. Oblicz modulo z większej liczby.

$$123 \mod 10$$

Rozwiązanie:

Szukamy reszty z dzielenia 123 przez 10.

$$123 = 12 \cdot 10 + 3$$

$$123 \mod 10 = 3$$

Odpowiedź:

$$3$$

Zadanie 3. Oblicz dodawanie modulo.

$$(25 + 38) \mod 7$$

Rozwiązanie:

$$25 \mod 7 = 4$$

$$38 \mod 7 = 3$$

$$(25 + 38) \mod 7 = (4 + 3) \mod 7$$

$$7 \mod 7 = 0$$

Odpowiedź:

$$0$$

Zadanie 4. Oblicz odejmowanie modulo.

$$(3 - 5) \mod 7$$

Rozwiązanie:

$$3 - 5 = -2$$

Wynik ujemny sprowadzamy do dodatniego wyniku modulo 7:

$$-2 + 7 = 5$$

Odpowiedź:

$$(3 - 5) \mod 7 = 5$$

Zadanie 5. Oblicz mnożenie modulo.

$$(13 \cdot 17) \mod 5$$

Rozwiązanie:

Najpierw redukujemy obie liczby modulo 5:

$$13 \mod 5 = 3$$

$$17 \mod 5 = 2$$

Teraz mnożymy reszty:

$$(13 \cdot 17) \mod 5 = (3 \cdot 2) \mod 5$$

$$6 \mod 5 = 1$$

Odpowiedź:

$$1$$

Zadanie 6. Uprość wyrażenie modulo.

$$(42 \cdot 19 + 15) \mod 8$$

Rozwiązanie:

Redukujemy liczby modulo 8:

$$42 \mod 8 = 2$$

$$19 \mod 8 = 3$$

$$15 \mod 8 = 7$$

Podstawiamy:

$$(42 \cdot 19 + 15) \mod 8 = (2 \cdot 3 + 7) \mod 8$$

$$(6 + 7) \mod 8 = 13 \mod 8 = 5$$

Odpowiedź:

$$5$$

Zadanie 7. Oblicz potęgę modulo prostą metodą.

$$3^4 \mod 5$$

Rozwiązanie:

$$3^2 = 9$$

$$9 \mod 5 = 4$$

$$3^4 = (3^2)^2$$

$$3^4 \mod 5 = 4^2 \mod 5 = 16 \mod 5 = 1$$

Odpowiedź:

$$1$$

Zadanie 8. Oblicz wyrażenie z ujemną resztą.

$$(-17) \mod 5$$

Rozwiązanie:

Szukamy liczby z zakresu $0, \dots, 4$.

$$-17 + 20 = 3$$

bo 20 jest wielokrotnością 5.

$$-17 \mod 5 = 3$$

Odpowiedź:

$$3$$

Zadanie 9. Sprawdź kongruencję.

Czy zachodzi?

$$27 \equiv 6 \pmod{7}$$

Rozwiązanie:

Sprawdzamy reszty:

$$27 \bmod 7 = 6$$

$$6 \bmod 7 = 6$$

Reszty są takie same, więc kongruencja zachodzi.

Odpowiedź:

$$27 \equiv 6 \pmod{7}$$

jest prawdą.

Zadanie 10. Prawda czy fałsz?

$$(A \cdot B) \bmod n = ((A \bmod n)(B \bmod n)) \bmod n$$

Odpowiedź: Prawda.

Uzasadnienie:

W mnożeniu modulo można redukować liczby przed wykonaniem działania.

Zadanie 11. Prawda czy fałsz?

Dzielenie modulo działa tak samo jak zwykłe dzielenie.

Odpowiedź: Fałsz.

Uzasadnienie:

W arytmetyce modularnej dzielenie zastępuje się mnożeniem przez odwrotność modularną, jeśli taka odwrotność istnieje.

Zadanie 12. Prawda czy fałsz?

Można redukować liczby modulo po każdym kroku obliczeń.

Odpowiedź: Prawda.

Uzasadnienie:

Własności modulo pozwalają zastępować duże liczby ich resztami, np.:

$$(A \cdot B) \bmod n = ((A \bmod n)(B \bmod n)) \bmod n$$

Dzięki temu obliczenia są krótsze i wygodniejsze.

Zadanie 13. Prawda czy fałsz?

Jeżeli wynik działania modulo jest ujemny, można dodać modulo, aby dostać wynik dodatni.

Odpowiedź: Prawda.

Przykład:

$$-2 \bmod 7 = 5$$

bo:

$$-2 + 7 = 5$$

Zadanie 14. Do czego przydaje się arytmetyka modularna?

Odpowiedź:

Arytmetyka modularna jest podstawą wielu algorytmów kryptograficznych, m.in.:

- RSA,
- Diffie–Hellman,
- ElGamal,
- funkcje hashujące.

Pozwala wykonywać obliczenia na dużych liczbach, ale utrzymywać wyniki w ograniczonym zakresie.

1.2 Szybkie potęgowanie modularne

Definicja

Szybkie potęgowanie modularne to metoda obliczania wartości:

$$a^b \mod n$$

bez liczenia ogromnej liczby a^b bezpośrednio.

Metoda polega na rozbiciu wykładnika b na potęgi dwójki i redukowaniu wyniku modulo po każdym kroku.

Używana jest m.in. w RSA, Diffie–Hellmanie i ElGamalu.

Najważniejsze wzory

Podstawowy problem:

$$a^b \mod n$$

Rozkład wykładnika na potęgi dwójki:

$$b = 2^{k_1} + 2^{k_2} + \dots + 2^{k_m}$$

Wtedy:

$$a^b = a^{2^{k_1}} \cdot a^{2^{k_2}} \cdot \dots \cdot a^{2^{k_m}}$$

Po każdym mnożeniu bierzemy modulo n .

Ważna własność:

$$(A \cdot B) \mod n = ((A \mod n)(B \mod n)) \mod n$$

Własności i zasady

- Nie liczymy a^b bezpośrednio.
- Wykładnik rozkładamy na potęgi dwójki.
- Liczymy kolejno:

$$a^1, a^2, a^4, a^8, \dots$$

zawsze modulo n .

- Do końcowego wyniku mnożymy tylko te potęgi, które występują w rozkładzie wykładnika.
- Po każdym mnożeniu redukujemy wynik modulo.
- Metoda ma złożoność logarytmiczną względem wykładnika.

Metoda krok po kroku

Jak liczyć $a^b \bmod n$ ręcznie:

1. Zapisz wykładnik b jako sumę potęg dwójki.

2. Policz kolejne potęgi:

$$a^1 \bmod n, \quad a^2 \bmod n, \quad a^4 \bmod n, \quad a^8 \bmod n, \dots$$

3. Wybierz tylko te potęgi, które występują w rozkładzie b .

4. Pomnóż je przez siebie.

5. Po każdym mnożeniu wykonuj modulo n .

Schemat:

$$a^b \bmod n$$

$$b = 2^k + 2^j + \dots$$

$$a^b \bmod n = (a^{2^k} \cdot a^{2^j} \cdot \dots) \bmod n$$

Przykład z zajęć

Oblicz:

$$2^9 \bmod 10$$

Rozkład wykładnika:

$$9 = 8 + 1$$

czyli:

$$2^9 = 2^8 \cdot 2^1$$

Liczymy kolejne potęgi modulo 10:

$$2^1 \bmod 10 = 2$$

$$2^2 \bmod 10 = 4$$

$$2^4 \bmod 10 = 4^2 \bmod 10 = 16 \bmod 10 = 6$$

$$2^8 \bmod 10 = 6^2 \bmod 10 = 36 \bmod 10 = 6$$

Składamy wynik:

$$2^9 \mod 10 = (2^8 \cdot 2^1) \mod 10$$

$$(6 \cdot 2) \mod 10 = 12 \mod 10 = 2$$

Odpowiedź:

$$2^9 \mod 10 = 2$$

Typowe pułapki

- Nie wolno liczyć ogromnej potęgi bezpośrednio.
- Trzeba redukować modulo po każdym kroku.
- Nie mnożymy wszystkich potęg, tylko te, które występują w rozkładzie wykładnika.
- Łatwo pomylić rozkład wykładnika, np.:

$$41 = 32 + 8 + 1$$

a nie $32 + 4 + 1$.

- Wynik końcowy również trzeba zredukować modulo.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Oblicz:

$$2^9 \mod 10$$

Rozwiązanie:

$$9 = 8 + 1$$

$$2^1 \mod 10 = 2$$

$$2^2 \mod 10 = 4$$

$$2^4 \mod 10 = 4^2 \mod 10 = 16 \mod 10 = 6$$

$$2^8 \mod 10 = 6^2 \mod 10 = 36 \mod 10 = 6$$

$$2^9 \mod 10 = (2^8 \cdot 2^1) \mod 10$$

$$(6 \cdot 2) \mod 10 = 12 \mod 10 = 2$$

Odpowiedź:

$$2$$

Zadanie 2. Oblicz:

$$2^{41} \bmod 10$$

Rozwiązanie:

Rozkład wykładnika:

$$41 = 32 + 8 + 1$$

czyli:

$$2^{41} = 2^{32} \cdot 2^8 \cdot 2^1$$

Liczymy kolejne potęgi modulo 10:

$$2^1 \bmod 10 = 2$$

$$2^2 \bmod 10 = 4$$

$$2^4 \bmod 10 = 4^2 \bmod 10 = 16 \bmod 10 = 6$$

$$2^8 \bmod 10 = 6^2 \bmod 10 = 36 \bmod 10 = 6$$

$$2^{16} \bmod 10 = 6^2 \bmod 10 = 36 \bmod 10 = 6$$

$$2^{32} \bmod 10 = 6^2 \bmod 10 = 36 \bmod 10 = 6$$

Składamy wynik:

$$2^{41} \bmod 10 = (2^{32} \cdot 2^8 \cdot 2^1) \bmod 10$$

$$(6 \cdot 6 \cdot 2) \bmod 10$$

$$36 \bmod 10 = 6$$

$$(6 \cdot 2) \bmod 10 = 12 \bmod 10 = 2$$

Odpowiedź:

$$2$$

Zadanie 3. Oblicz:

$$3^{13} \mod 17$$

Rozwiązanie:

Rozkład wykładnika:

$$13 = 8 + 4 + 1$$

Liczymy potęgi modulo 17:

$$3^1 \mod 17 = 3$$

$$3^2 \mod 17 = 9$$

$$3^4 \mod 17 = 9^2 \mod 17 = 81 \mod 17 = 13$$

$$3^8 \mod 17 = 13^2 \mod 17 = 169 \mod 17 = 16$$

Składamy wynik:

$$3^{13} \mod 17 = (3^8 \cdot 3^4 \cdot 3^1) \mod 17$$

$$(16 \cdot 13 \cdot 3) \mod 17$$

$$16 \cdot 13 = 208$$

$$208 \mod 17 = 4$$

$$4 \cdot 3 = 12$$

$$12 \mod 17 = 12$$

Odpowiedź:

$$12$$

Zadanie 4. Oblicz:

$$5^{11} \mod 23$$

Rozwiązanie:

$$11 = 8 + 2 + 1$$

$$5^1 \mod 23 = 5$$

$$5^2 \mod 23 = 25 \mod 23 = 2$$

$$5^4 \mod 23 = 2^2 \mod 23 = 4$$

$$5^8 \mod 23 = 4^2 \mod 23 = 16$$

$$5^{11} \mod 23 = (5^8 \cdot 5^2 \cdot 5^1) \mod 23$$

$$(16 \cdot 2 \cdot 5) \mod 23$$

$$32 \mod 23 = 9$$

$$9 \cdot 5 = 45$$

$$45 \mod 23 = 22$$

Odpowiedź:

$$22$$

Zadanie 5. Prawda czy fałsz?

Szybkie potęgowanie modularne polega na obliczeniu całej liczby a^b , a dopiero potem wykonaniu modulo.

Odpowiedź: Fałsz.

Uzasadnienie:

W szybkim potęgowaniu modularnym redukujemy wynik modulo po każdym kroku, żeby nie operować na ogromnych liczbach.

Zadanie 6. Prawda czy fałsz?

W szybkim potęgowaniu modularnym wykładnik rozkładamy na potęgi dwójki.

Odpowiedź: Prawda.

Uzasadnienie:

Na przykład:

$$41 = 32 + 8 + 1$$

więc:

$$a^{41} = a^{32} \cdot a^8 \cdot a^1$$

Zadanie 7. Do czego używa się szybkiego potęgowania modularnego?

Odpowiedź:

Szybkie potęgowanie modularne jest używane m.in. w:

- RSA,
- Diffie–Hellmanie,
- ElGamalu.

Pozwala szybko obliczać wyrażenia postaci:

$$a^b \mod n$$

bez liczenia ogromnej potęgi a^b bezpośrednio.

1.3 Rozszerzony algorytm Euklidesa

Definicja

Rozszerzony algorytm Euklidesa służy do:

- obliczania największego wspólnego dzielnika $NWD(a, b)$,
- znajdowania współczynników równania:

$$ax + by = NWD(a, b)$$

- wyznaczania odwrotności modularnej.

Jest bardzo ważny w RSA, ElGamalu i arytmetyce modularnej.
Na zajęciach prowadzący wykonywał go tabelką.

Najważniejsze wzory

Podstawowy krok algorytmu:

$$a = bq + r$$

gdzie:

- a – większa liczba,
- b – mniejsza liczba,
- q – iloraz całkowity,
- r – reszta.

Algorytm kończy się, gdy:

$$r = 0$$

Ostatnia niezerowa reszta to:

$$NWD(a, b)$$

Rozszerzona postać:

$$ax + by = NWD(a, b)$$

Własności i zasady

- Algorytm działa poprzez kolejne dzielenia z resztą.
- Każdy krok zmniejsza liczby.
- Ostatnia niezerowa reszta jest największym wspólnym dzielnikiem.
- Rozszerzona wersja pozwala znaleźć współczynniki x i y .
- Jeśli:

$$NWD(a, n) = 1$$

to istnieje odwrotność modularna liczby a modulo n .

Metoda krok po kroku

Jak wykonać rozszerzony algorytm Euklidesa ręcznie:

1. Dziel większą liczbę przez mniejszą.
2. Zapisz:

$$a = bq + r$$

3. Powtarzaj dla:

$$b, r$$

aż otrzymasz resztę 0.

4. Ostatnia niezerowa reszta to NWD .
5. Cofając obliczenia wyznacz współczynniki x i y .

Schemat tabelki z zajęć:

$$a = bq + r$$

$$b = rq + r$$

...

aż do:

$$r = 0$$

Przykład z zajęć

Oblicz:

$$NWD(35, 12)$$

Dzielimy:

$$35 = 12 \cdot 2 + 11$$

$$12 = 11 \cdot 1 + 1$$

$$11 = 1 \cdot 11 + 0$$

Ostatnia niezerowa reszta:

$$1$$

czyli:

$$NWD(35, 12) = 1$$

Cofamy podstawienia:

$$1 = 12 - 11 \cdot 1$$

ale:

$$11 = 35 - 12 \cdot 2$$

Podstawiamy:

$$1 = 12 - (35 - 12 \cdot 2)$$

$$1 = 12 - 35 + 24$$

$$1 = 3 \cdot 12 - 35$$

czyli:

$$1 = (-1) \cdot 35 + 3 \cdot 12$$

Współczynniki:

$$x = -1$$

$$y = 3$$

Typowe pułapki

- Łatwo pomylić iloraz q i resztę r .
- Ostatnia niezerowa reszta to NWD , a nie ostatnia liczba.
- Przy cofaniach łatwo zgubić znaki.
- Trzeba poprawnie podstawiać wcześniejsze równania.
- Jeśli:

$$NWD(a, n) \neq 1$$

to odwrotność modularna NIE istnieje.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Oblicz:

$$NWD(35, 12)$$

Rozwiązanie:

$$35 = 12 \cdot 2 + 11$$

$$12 = 11 \cdot 1 + 1$$

$$11 = 1 \cdot 11 + 0$$

Ostatnia niezerowa reszta:

$$1$$

Odpowiedź:

$$NWD(35, 12) = 1$$

Zadanie 2. Znajdź współczynniki x, y :

$$35x + 12y = 1$$

Rozwiązanie:

Z poprzedniego przykładu:

$$1 = 12 - 11$$

$$11 = 35 - 12 \cdot 2$$

Podstawiamy:

$$1 = 12 - (35 - 12 \cdot 2)$$

$$1 = 3 \cdot 12 - 35$$

czyli:

$$1 = (-1) \cdot 35 + 3 \cdot 12$$

Odpowiedź:

$$x = -1$$

$$y = 3$$

Zadanie 3. Oblicz:

$$NWD(48, 18)$$

Rozwiązanie:

$$48 = 18 \cdot 2 + 12$$

$$18 = 12 \cdot 1 + 6$$

$$12 = 6 \cdot 2 + 0$$

Ostatnia niezerowa reszta:

$$6$$

Odpowiedź:

$$NWD(48, 18) = 6$$

Zadanie 4. Czy istnieje odwrotność modułarna liczby 12 mod 18?

Rozwiązanie:

Sprawdzamy:

$$NWD(12, 18)$$

$$18 = 12 \cdot 1 + 6$$

$$12 = 6 \cdot 2 + 0$$

$$NWD(12, 18) = 6$$

Ponieważ:

$$6 \neq 1$$

odwrotność modularna nie istnieje.

Odpowiedź:

Nie istnieje.

Zadanie 5. Prawda czy fałsz?

Rozszerzony algorytm Euklidesa można wykorzystać do wyznaczania odwrotności modularnej.

Odpowiedź: Prawda.

Uzasadnienie:

Algorytm pozwala znaleźć współczynniki:

$$ax + ny = 1$$

Współczynnik x jest odwrotnością modularną liczby a .

Zadanie 6. Prawda czy fałsz?

Jeżeli:

$$NWD(a, n) \neq 1$$

to odwrotność modularna istnieje.

Odpowiedź: Fałsz.

Uzasadnienie:

Odwrotność modularna istnieje tylko wtedy, gdy:

$$NWD(a, n) = 1$$

Zadanie 7. Do czego używa się rozszerzonego algorytmu Euklidesa?

Odpowiedź:

Rozszerzony algorytm Euklidesa służy do:

- obliczania NWD ,
- wyznaczania współczynników równania:

$$ax + by = NWD(a, b)$$

- znajdowania odwrotności modularnej,
- zastosowań w RSA i ElGamalu.

1.4 Odwrotność modularna

Definicja

Odwrotność modularna liczby a modulo n to taka liczba x , że:

$$a \cdot x \equiv 1 \pmod{n}$$

Oznacza to, że po pomnożeniu liczby przez jej odwrotność modularną otrzymujemy resztę 1 modulo n .

Odwrotność modularna jest używana m.in. w:

- RSA,
- ElGamalu,
- dzieleniu modulo,
- kryptografii asymetrycznej.

Najważniejsze wzory

Definicja odwrotności modularnej:

$$a^{-1} \pmod{n}$$

spełnia:

$$a \cdot a^{-1} \equiv 1 \pmod{n}$$

Odwrotność istnieje tylko wtedy, gdy:

$$\text{NWD}(a, n) = 1$$

Rozszerzony algorytm Euklidesa:

$$ax + ny = 1$$

Współczynnik x jest odwrotnością modularną liczby a .

Własności i zasady

- Nie każda liczba ma odwrotność modularną.
- Odwrotność istnieje tylko wtedy, gdy:

$$\text{NWD}(a, n) = 1$$

- Dzielenie modulo zastępujemy mnożeniem przez odwrotność modularną.
- Odwrotność można znaleźć:
 - brute force,
 - rozszerzonym algorytmem Euklidesa.
- Na zajęciach prowadzący wymagał metody z algorytmem Euklidesa.

Metoda krok po kroku

Jak znaleźć odwrotność modularną:

1. Sprawdź:

$$\text{NWD}(a, n)$$

2. Jeśli:

$$\text{NWD}(a, n) \neq 1$$

odwrotność nie istnieje.

3. Wykonaj rozszerzony algorytm Euklidesa.

4. Znajdź współczynniki:

$$ax + ny = 1$$

5. Współczynnik x jest odwrotnością modularną.

6. Jeśli wynik jest ujemny, dodaj modulo n .

Schemat z zajęć:

$$21x \equiv 1 \pmod{8}$$

Szukamy:

$$21x + 8y = 1$$

Przykład z zajęć

Znajdź odwrotność modularną:

$$21^{-1} \pmod{8}$$

Rozwiązujemy:

$$21x + 8y = 1$$

Dzielimy:

$$21 = 8 \cdot 2 + 5$$

$$8 = 5 \cdot 1 + 3$$

$$5 = 3 \cdot 1 + 2$$

$$3 = 2 \cdot 1 + 1$$

$$2 = 1 \cdot 2 + 0$$

Ostatnia niezerowa reszta:

$$1$$

czyli odwrotność istnieje.

Cofamy podstawienia:

$$1 = 3 - 2$$

$$2 = 5 - 3$$

$$1 = 3 - (5 - 3)$$

$$1 = 2 \cdot 3 - 5$$

$$3 = 8 - 5$$

$$1 = 2(8 - 5) - 5$$

$$1 = 2 \cdot 8 - 3 \cdot 5$$

$$5 = 21 - 2 \cdot 8$$

$$1 = 2 \cdot 8 - 3(21 - 2 \cdot 8)$$

$$1 = 8 \cdot 8 - 3 \cdot 21$$

czyli:

$$1 = (-3) \cdot 21 + 8 \cdot 8$$

Współczynnik przy 21:

$$-3$$

Dodajemy modulo 8:

$$-3 + 8 = 5$$

Odpowiedź:

$$21^{-1} \mod 8 = 5$$

Typowe pułapki

- Odwrotność modularna istnieje tylko wtedy, gdy:

$$\text{NWD}(a, n) = 1$$

- Łatwo zgubić znaki podczas cofania podstawień.
- Ujemny wynik trzeba sprowadzić do dodatniego modulo.
- Nie wolno używać zwykłego dzielenia modulo.
- Prowadzący wymagał metody z rozszerzonym algorytmem Euklidesa, a nie brute force.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Znajdź odwrotność modularną:

$$3^{-1} \pmod{7}$$

Rozwiązanie:

Szukamy liczby x , dla której:

$$3x \equiv 1 \pmod{7}$$

Sprawdzamy:

$$3 \cdot 5 = 15$$

$$15 \pmod{7} = 1$$

Odpowiedź:

$$3^{-1} \pmod{7} = 5$$

Zadanie 2. Znajdź odwrotność modularną:

$$21^{-1} \pmod{8}$$

Rozwiązanie:

Z rozszerzonego algorytmu Euklidesa:

$$1 = (-3) \cdot 21 + 8 \cdot 8$$

Współczynnik przy 21:

$$-3$$

Sprowadzamy do dodatniego modulo:

$$-3 + 8 = 5$$

Odpowiedź:

$$21^{-1} \pmod{8} = 5$$

Zadanie 3. Czy istnieje odwrotność modularna liczby $12 \pmod{18}$?

Rozwiązanie:

Sprawdzamy:

$$NWD(12, 18)$$

$$18 = 12 \cdot 1 + 6$$

$$12 = 6 \cdot 2 + 0$$

$$NWD(12, 18) = 6$$

Ponieważ:

$$6 \neq 1$$

odwrotność modularna nie istnieje.

Odpowiedź:

Nie istnieje.

Zadanie 4. Oblicz:

$$5^{-1} \pmod{11}$$

Rozwiązanie:

Szukamy:

$$5x \equiv 1 \pmod{11}$$

Sprawdzamy:

$$5 \cdot 9 = 45$$

$$45 \pmod{11} = 1$$

Odpowiedź:

$$5^{-1} \pmod{11} = 9$$

Zadanie 5. Prawda czy fałsz?

Jeżeli:

$$\text{NWD}(a, n) = 1$$

to odwrotność modularna istnieje.

Odpowiedź: Prawda.

Uzasadnienie:

To podstawowy warunek istnienia odwrotności modularnej.

Zadanie 6. Prawda czy fałsz?

Każda liczba ma odwrotność modularną.

Odpowiedź: Fałsz.

Uzasadnienie:

Odwrotność istnieje tylko wtedy, gdy:

$$\text{NWD}(a, n) = 1$$

Zadanie 7. Do czego używa się odwrotności modularnej?

Odpowiedź:

Odwrotność modularna jest używana m.in. w:

- RSA,
- ElGamalu,
- dzieleniu modulo,
- kryptografii asymetrycznej.

Pozwala zastąpić dzielenie modulo mnożeniem.

1.5 NWD / gcd

Definicja

NWD (Największy Wspólny Dzielnik), czyli gcd (*greatest common divisor*), to największa liczba dzieląca jednocześnie dwie liczby bez reszty.

Przykład:

$$NWD(12, 18) = 6$$

ponieważ 6 jest największą liczbą dzielącą jednocześnie 12 i 18.

NWD jest bardzo ważny w:

- arytmetyce modularnej,
- RSA,
- odwrotności modularnej,
- testach pierwszości.

Najważniejsze wzory

Definicja:

$$NWD(a, b)$$

Algorytm Euklidesa:

$$a = bq + r$$

gdzie:

- a – większa liczba,
- b – mniejsza liczba,
- q – iloraz,
- r – reszta.

Algorytm kończy się, gdy:

$$r = 0$$

Ostatnia niezerowa reszta to:

$$NWD(a, b)$$

Własności i zasady

- Jeżeli:

$$NWD(a, b) = 1$$

to liczby są względnie pierwsze.

- Odwrotność modularna istnieje tylko wtedy, gdy:

$$NWD(a, n) = 1$$

- Liczba pierwsza ma NWD równe 1 z każdą liczbą, która nie jest jej wielokrotnością.
- Algorytm Euklidesa jest bardzo szybki nawet dla dużych liczb.

Metoda krok po kroku

Jak obliczyć NWD metodą Euklidesa:

1. Podziel większą liczbę przez mniejszą.
2. Zapisz:

$$a = bq + r$$

3. Zamień:

$$(a, b) \rightarrow (b, r)$$

4. Powtarzaj aż:

$$r = 0$$

5. Ostatnia niezerowa reszta to NWD.

Schemat:

$$48 = 18 \cdot 2 + 12$$

$$18 = 12 \cdot 1 + 6$$

$$12 = 6 \cdot 2 + 0$$

$$NWD = 6$$

Przykład z zajęć

Oblicz:

$$NWD(48, 18)$$

Dzielimy:

$$48 = 18 \cdot 2 + 12$$

$$18 = 12 \cdot 1 + 6$$

$$12 = 6 \cdot 2 + 0$$

Ostatnia niezerowa reszta:

$$6$$

Odpowiedź:

$$NWD(48, 18) = 6$$

Sprawdzenie dzielników:

$$48 : 1, 2, 3, 4, 6, 8, 12, 16, 24, 48$$

$$18 : 1, 2, 3, 6, 9, 18$$

Największy wspólny dzielnik:

$$6$$

Typowe pułapki

- Ostatnia liczba przed zerem to NWD.
- Łatwo pomylić resztę r z ilorazem q .
- Nie kończymy algorytmu zbyt wcześnie.
- Jeśli:

$$NWD(a, n) \neq 1$$

to odwrotność modularna nie istnieje.

- Liczby względnie pierwsze NIE muszą być liczbami pierwszymi.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Oblicz:

$$NWD(48, 18)$$

Rozwiązanie:

$$48 = 18 \cdot 2 + 12$$

$$18 = 12 \cdot 1 + 6$$

$$12 = 6 \cdot 2 + 0$$

Ostatnia niezerowa reszta:

$$6$$

Odpowiedź:

$$NWD(48, 18) = 6$$

Zadanie 2. Oblicz:

$$NWD(35, 12)$$

Rozwiązanie:

$$35 = 12 \cdot 2 + 11$$

$$12 = 11 \cdot 1 + 1$$

$$11 = 1 \cdot 11 + 0$$

Ostatnia niezerowa reszta:

$$1$$

Odpowiedź:

$$NWD(35, 12) = 1$$

Liczby są względnie pierwsze.

Zadanie 3. Czy istnieje odwrotność modułarna liczby 12 mod 18?

Rozwiązanie:

Sprawdzamy:

$$NWD(12, 18)$$

$$18 = 12 \cdot 1 + 6$$

$$12 = 6 \cdot 2 + 0$$

$$NWD(12, 18) = 6$$

Ponieważ:

$$6 \neq 1$$

odwrotność modularna nie istnieje.

Odpowiedź:

Nie istnieje.

Zadanie 4. Prawda czy fałsz?

Jeżeli:

$$NWD(a, b) = 1$$

to liczby są względnie pierwsze.

Odpowiedź: Prawda.

Uzasadnienie:

Definicja liczb względnie pierwszych.

Zadanie 5. Prawda czy fałsz?

Każde dwie liczby pierwsze są względnie pierwsze.

Odpowiedź: Prawda.

Uzasadnienie:

Dwie różne liczby pierwsze mają tylko jeden wspólny dzielnik:

$$1$$

Zadanie 6. Prawda czy fałsz?

Jeżeli:

$$NWD(a, n) \neq 1$$

to odwrotność modularna istnieje.

Odpowiedź: Fałsz.

Uzasadnienie:

Odwrotność modularna istnieje tylko wtedy, gdy:

$$NWD(a, n) = 1$$

Zadanie 7. Do czego używa się NWD?

Odpowiedź:

NWD jest używany m.in. w:

- RSA,
- odwrotności modularnej,
- arytmetyce modularnej,
- testach pierwszości,
- rozszerzonym algorytmie Euklidesa.

1.6 Funkcja Eulera $\varphi(n)$

Definicja

Funkcja Eulera $\varphi(n)$ określa liczbę dodatnich liczb mniejszych od n , które są względnie pierwsze z n .

Oznacza to liczby, dla których:

$$NWD(k, n) = 1$$

Funkcja Eulera jest bardzo ważna w:

- RSA,
- arytmetyce modularnej,
- odwrotności modularnej,
- twierdzeniu Eulera.

Najważniejsze wzory

Dla liczby pierwszej:

$$\varphi(p) = p - 1$$

ponieważ każda liczba mniejsza od p jest względnie pierwsza z p .

Dla iloczynu dwóch różnych liczb pierwszych:

$$\varphi(pq) = (p - 1)(q - 1)$$

To najważniejszy wzór używany w RSA.

Ogólna definicja:

$$\varphi(n) = |\{k < n : NWD(k, n) = 1\}|$$

Własności i zasady

- Funkcja Eulera liczy liczby względnie pierwsze z n .
- Jeśli p jest liczbą pierwszą:

$$\varphi(p) = p - 1$$

- W RSA bardzo często używa się:

$$\varphi(n) = (p - 1)(q - 1)$$

gdzie:

$$n = pq$$

- Funkcja Eulera jest potrzebna do wyznaczenia klucza prywatnego RSA.
- Liczby względnie pierwsze mają:

$$NWD(a, n) = 1$$

Metoda krok po kroku

Jak policzyć funkcję Eulera:

Metoda 1 — ręczne sprawdzanie

1. Wypisz liczby mniejsze od n .
2. Dla każdej liczby sprawdź:

$$NWD(k, n)$$

3. Policz liczby względnie pierwsze z n .

Metoda 2 — wzór RSA

Jeżeli:

$$n = pq$$

oraz p, q są pierwsze:

$$\varphi(n) = (p - 1)(q - 1)$$

Przykład z zajęć

Oblicz:

$$\varphi(10)$$

Wypisujemy liczby mniejsze od 10:

$$1, 2, 3, 4, 5, 6, 7, 8, 9$$

Sprawdzamy NWD z 10:

$$NWD(1, 10) = 1$$

$$NWD(2, 10) = 2$$

$$NWD(3, 10) = 1$$

$$NWD(4, 10) = 2$$

$$NWD(5, 10) = 5$$

$$NWD(6, 10) = 2$$

$$NWD(7, 10) = 1$$

$$NWD(8, 10) = 2$$

$$NWD(9, 10) = 1$$

Względnie pierwsze z 10:

$$1, 3, 7, 9$$

Są 4 takie liczby.

Odpowiedź:

$$\varphi(10) = 4$$

Przykład RSA:

$$p = 5, \quad q = 11$$

$$n = 55$$

$$\varphi(55) = (5 - 1)(11 - 1)$$

$$\varphi(55) = 4 \cdot 10 = 40$$

Typowe pułapki

- $\varphi(n)$ nie oznacza liczby pierwszej.
- W RSA:

$$\varphi(n) \neq n - 1$$

gdy:

$$n = pq$$

- Łatwo pomylić liczby pierwsze z liczbami względnie pierwszymi.
- Wzór:

$$\varphi(pq) = (p-1)(q-1)$$

działa dla dwóch różnych liczb pierwszych.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Oblicz:

$$\varphi(7)$$

Rozwiązanie:

7 jest liczbą pierwszą.

Dla liczby pierwszej:

$$\varphi(p) = p - 1$$

czyli:

$$\varphi(7) = 6$$

Odpowiedź:

$$6$$

Zadanie 2. Oblicz:

$$\varphi(10)$$

Rozwiązanie:

Liczby względnie pierwsze z 10:

$$1, 3, 7, 9$$

Są 4 takie liczby.

Odpowiedź:

$$\varphi(10) = 4$$

Zadanie 3. Oblicz:

$$\varphi(55)$$

gdzie:

$$55 = 5 \cdot 11$$

Rozwiązanie:

$$\varphi(55) = (5-1)(11-1)$$

$$\varphi(55) = 4 \cdot 10 = 40$$

Odpowiedź:

40

Zadanie 4. Prawda czy fałsz?

Jeżeli p jest liczbą pierwszą:

$$\varphi(p) = p - 1$$

Odpowiedź: Prawda.

Uzasadnienie:

Każda liczba mniejsza od liczby pierwszej jest z nią względnie pierwsza.

Zadanie 5. Prawda czy fałsz?

W RSA używa się funkcji Eulera.

Odpowiedź: Prawda.

Uzasadnienie:

Funkcja Eulera jest potrzebna do wyznaczenia klucza prywatnego.

Zadanie 6. Prawda czy fałsz?

$$\varphi(10) = 9$$

Odpowiedź: Fałsz.

Uzasadnienie:

Liczby względnie pierwsze z 10:

1, 3, 7, 9

czyli:

$$\varphi(10) = 4$$

Zadanie 7. Do czego używa się funkcji Eulera?

Odpowiedź:

Funkcja Eulera jest używana m.in. w:

- RSA,
- arytmetyce modularnej,
- odwrotności modularnej,
- twierdzeniu Eulera.

1.7 Trial Division (metoda próbnego dzielenia)

Definicja

Trial Division (metoda próbnego dzielenia) to najprostsza metoda sprawdzania, czy liczba jest pierwsza.

Polega na sprawdzaniu, czy dana liczba ma dzielniki inne niż:

$$1$$

oraz ona sama.

Jeżeli znajdziemy dzielnik, liczba NIE jest pierwsza.

Metoda była używana na zajęciach przy testach pierwszości oraz liczbach Mersenne'a.

Najważniejsze wzory

Liczba n jest pierwsza, jeśli:

$$n > 1$$

oraz nie istnieje liczba:

$$d$$

taka, że:

$$1 < d < n$$

i:

$$d \mid n$$

Najważniejsza własność:

Wystarczy sprawdzać dzielniki do:

$$\sqrt{n}$$

ponieważ jeśli:

$$n = a \cdot b$$

to przynajmniej jeden z dzielników spełnia:

$$a \leq \sqrt{n}$$

Własności i zasady

- Metoda próbnego dzielenia jest bardzo prosta, ale wolna dla dużych liczb.
- Sprawdzamy kolejne dzielniki:

$$2, 3, 4, 5, \dots$$

- Jeśli znajdziemy dzielnik:

$$d \mid n$$

to liczba NIE jest pierwsza.

- Wystarczy sprawdzać dzielniki do:

$$\sqrt{n}$$

- Liczba parzysta większa od 2 nie jest pierwsza.

Metoda krok po kroku

Jak sprawdzić pierwszość metodą próbnego dzielenia:

1. Sprawdź, czy liczba jest większa od 1.
2. Oblicz:

$$\sqrt{n}$$

3. Sprawdzaj kolejne liczby:

$$2, 3, 4, \dots, \sqrt{n}$$

4. Jeśli:

$$d \mid n$$

liczba NIE jest pierwsza.

5. Jeśli nie znaleziono dzielnika, liczba jest pierwsza.

Schemat z zajęć:

$$n = 37$$

$$\sqrt{37} \approx 6$$

Sprawdzamy:

$$2, 3, 4, 5, 6$$

Brak dzielników.

Liczba jest pierwsza.

Przykład z zajęć

Sprawdź, czy liczba:

$$37$$

jest pierwsza.

Liczymy:

$$\sqrt{37} \approx 6$$

Sprawdzamy dzielniki:

$$37 \bmod 2 \neq 0$$

$$37 \bmod 3 \neq 0$$

$$37 \bmod 4 \neq 0$$

$$37 \bmod 5 \neq 0$$

$$37 \bmod 6 \neq 0$$

Nie znaleziono dzielnika.

Odpowiedź:

$$37$$

jest liczbą pierwszą.

Sprawdźmy teraz:

$$35$$

$$35 \bmod 5 = 0$$

Znaleziono dzielnik.

Odpowiedź:

$$35$$

nie jest liczbą pierwszą.

Typowe pułapki

- Nie trzeba sprawdzać dzielników większych niż:

$$\sqrt{n}$$

- Liczba:

1

NIE jest liczbą pierwszą.

- Liczba parzysta większa od 2 nie jest pierwsza.
- Łatwo zapomnieć o sprawdzeniu liczby:

2

- Znalezienie jednego dzielnika oznacza, że liczba nie jest pierwsza.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Sprawdź, czy liczba 37 jest pierwsza.

Rozwiązanie:

$$\sqrt{37} \approx 6$$

Sprawdzamy:

2, 3, 4, 5, 6

$$37 \bmod 2 \neq 0$$

$$37 \bmod 3 \neq 0$$

$$37 \bmod 4 \neq 0$$

$$37 \bmod 5 \neq 0$$

$$37 \bmod 6 \neq 0$$

Nie znaleziono dzielnika.

Odpowiedź:

37

jest liczbą pierwszą.

Zadanie 2. Sprawdź, czy liczba 35 jest pierwsza.

Rozwiązanie:

$$35 \bmod 5 = 0$$

Znaleziono dzielnik:

$$5$$

Odpowiedź:

$$35$$

nie jest liczbą pierwszą.

Zadanie 3. Sprawdź, czy liczba 29 jest pierwsza.

Rozwiązanie:

$$\sqrt{29} \approx 5$$

Sprawdzamy:

$$2, 3, 4, 5$$

$$29 \bmod 2 \neq 0$$

$$29 \bmod 3 \neq 0$$

$$29 \bmod 4 \neq 0$$

$$29 \bmod 5 \neq 0$$

Nie znaleziono dzielnika.

Odpowiedź:

$$29$$

jest liczbą pierwszą.

Zadanie 4. Prawda czy fałsz?

W metodzie próbnego dzielenia trzeba sprawdzać dzielniki aż do:

$$n - 1$$

Odpowiedź: Fałsz.

Uzasadnienie:

Wystarczy sprawdzać dzielniki do:

$$\sqrt{n}$$

Zadanie 5. Prawda czy fałsz?

Liczba 1 jest liczbą pierwszą.

Odpowiedź: Fałsz.

Uzasadnienie:

Liczba pierwsza musi mieć dokładnie dwa dzielniki:

1

oraz samą siebie.

Zadanie 6. Prawda czy fałsz?

Każda liczba parzysta większa od 2 nie jest pierwsza.

Odpowiedź: Prawda.

Uzasadnienie:

Każda liczba parzysta dzieli się przez:

2

Zadanie 7. Do czego używa się trial division?

Odpowiedź:

Trial Division służy do:

- sprawdzania pierwszościc liczb,
- znajdowania dzielników,
- prostych testów kryptograficznych,
- nauki testów pierwszościc.

1.8 Test pierwszości liczb

Definicja

Test pierwszości służy do sprawdzania, czy dana liczba jest liczbą pierwszą. Liczba pierwsza to liczba większa od:

$$1$$

która ma dokładnie dwa dzielniki:

$$1$$

oraz samą siebie.

Testy pierwszości są bardzo ważne w kryptografii, szczególnie w:

- RSA,
- Diffie–Hellmanie,
- ElGamalu,
- generowaniu kluczy kryptograficznych.

Najważniejsze wzory

Definicja liczby pierwszej:

$$p > 1$$

oraz:

$$d \mid p$$

tylko dla:

$$d = 1$$

oraz:

$$d = p$$

Najważniejsza własność:

W trial division wystarczy sprawdzać dzielniki do:

$$\sqrt{n}$$

Dla liczb Mersenne’a:

$$M_p = 2^p - 1$$

stosuje się test Lucasa–Lehmera.

Własności i zasady

- Liczba:

$$1$$

nie jest liczbą pierwszą.

- Liczba:

$$2$$

jest jedyną parzystą liczbą pierwszą.

- Każda liczba parzysta większa od 2 nie jest pierwsza.
- Do małych liczb często używa się trial division.
- W kryptografii używa się szybszych testów pierwszości.
- Duże liczby pierwsze są podstawą RSA.

Metoda krok po kroku

Jak sprawdzić, czy liczba jest pierwsza:

1. Sprawdź, czy:

$$n > 1$$

2. Oblicz:

$$\sqrt{n}$$

3. Sprawdzaj kolejne dzielniki:

$$2, 3, 4, \dots, \sqrt{n}$$

4. Jeśli:

$$d \mid n$$

liczba nie jest pierwsza.

5. Jeśli nie znaleziono dzielnika, liczba jest pierwsza.

Schemat z zajęć:

$$n = 29$$

$$\sqrt{29} \approx 5$$

Sprawdzamy:

$$2, 3, 4, 5$$

Brak dzielników.

Liczba jest pierwsza.

Przykład z zajęć

Sprawdź, czy liczba:

$$29$$

jest pierwsza.

Liczmy:

$$\sqrt{29} \approx 5$$

Sprawdzamy dzielniki:

$$29 \bmod 2 \neq 0$$

$$29 \bmod 3 \neq 0$$

$$29 \bmod 4 \neq 0$$

$$29 \bmod 5 \neq 0$$

Nie znaleziono dzielnika.

Odpowiedź:

$$29$$

jest liczbą pierwszą.

Sprawdźmy teraz:

$$21$$

$$21 \bmod 3 = 0$$

Znaleziono dzielnik.

Odpowiedź:

$$21$$

nie jest liczbą pierwszą.

Typowe pułapki

- Liczba:

1

NIE jest pierwsza.

- Nie trzeba sprawdzać dzielników większych niż:

$$\sqrt{n}$$

- Łatwo zapomnieć o sprawdzeniu liczby:

2

- Jedna znaleziona podzielność oznacza, że liczba nie jest pierwsza.
- Liczby względnie pierwsze to nie to samo co liczby pierwsze.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Sprawdź, czy liczba 29 jest pierwsza.

Rozwiązanie:

$$\sqrt{29} \approx 5$$

Sprawdzamy:

2, 3, 4, 5

$$29 \bmod 2 \neq 0$$

$$29 \bmod 3 \neq 0$$

$$29 \bmod 4 \neq 0$$

$$29 \bmod 5 \neq 0$$

Nie znaleziono dzielnika.

Odpowiedź:

29

jest liczbą pierwszą.

Zadanie 2. Sprawdź, czy liczba 21 jest pierwsza.

Rozwiązanie:

$$21 \bmod 3 = 0$$

Znaleziono dzielnik:

$$3$$

Odpowiedź:

$$21$$

nie jest liczbą pierwszą.

Zadanie 3. Sprawdź, czy liczba 97 jest pierwsza.

Rozwiązanie:

$$\sqrt{97} \approx 9$$

Sprawdzamy:

$$2, 3, 4, 5, 6, 7, 8, 9$$

Brak dzielników.

Odpowiedź:

$$97$$

jest liczbą pierwszą.

Zadanie 4. Prawda czy fałsz?

Liczba 1 jest liczbą pierwszą.

Odpowiedź: Fałsz.

Uzasadnienie:

Liczba pierwsza musi mieć dokładnie dwa dzielniki.

Zadanie 5. Prawda czy fałsz?

Liczba 2 jest jedyną parzystą liczbą pierwszą.

Odpowiedź: Prawda.

Uzasadnienie:

Każda inna liczba parzysta dzieli się przez:

$$2$$

Zadanie 6. Prawda czy fałsz?

W trial division trzeba sprawdzać dzielniki aż do:

$$n - 1$$

Odpowiedź: Fałsz.

Uzasadnienie:

Wystarczy sprawdzać dzielniki do:

$$\sqrt{n}$$

Zadanie 7. Do czego używa się testów pierwszości?

Odpowiedź:

Testy pierwszości są używane m.in. w:

- RSA,
- Diffie–Hellmanie,
- ElGamalu,
- generowaniu kluczy kryptograficznych.

1.9 Sito Eratostenesa

Definicja

Sito Eratostenesa to algorytm służący do znajdowania wszystkich liczb pierwszych mniejszych od zadanej liczby n .

Metoda polega na wykreślaniu kolejnych wielokrotności liczb pierwszych.

Jest to jeden z najstarszych i najprostszych algorytmów do generowania liczb pierwszych.

Najważniejsze wzory

Liczba pierwsza ma dokładnie dwa dzielniki:

$$1$$

oraz:

$$p$$

W sicie Eratostenesa wykreślamy wielokrotności:

$$2p, 3p, 4p, \dots$$

dla kolejnych liczb pierwszych p .

Wystarczy sprawdzać liczby do:

$$\sqrt{n}$$

Własności i zasady

- Sito Eratostenesa znajduje wszystkie liczby pierwsze z danego zakresu.
- Zaczynamy od liczby:

$$2$$

- Wykreślamy wszystkie wielokrotności liczby pierwszej.
- Niewykreślone liczby są pierwsze.
- Liczba:

$$1$$

nie jest pierwsza.

- Algorytm jest dużo szybszy niż trial division dla wielu liczb.

Metoda krok po kroku

Jak działa sito Eratostenesa:

1. Wypisz liczby od:

2

do n .

2. Pierwsza niewykreślona liczba jest pierwsza.
3. Wykreśl wszystkie jej wielokrotności.
4. Przejdź do kolejnej niewykreślonej liczby.
5. Powtarzaj aż do:

$\sqrt{n}$

Schemat z zajęć dla $n = 20$:

Wykreślamy wielokrotności:

2 : 4, 6, 8, 10, 12, 14, 16, 18, 20

3 : 6, 9, 12, 15, 18

5 : 10, 15, 20

Pozostałe liczby są pierwsze.

Przykład z zajęć

Znajdź liczby pierwsze mniejsze od:

20

Wypisujemy liczby:

2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20

Wykreślamy wielokrotności 2:

4, 6, 8, 10, 12, 14, 16, 18, 20

Wykreślamy wielokrotności 3:

6, 9, 12, 15, 18

Wykreślamy wielokrotności 5:

10, 15, 20

Pozostałe liczby:

2, 3, 5, 7, 11, 13, 17, 19

Odpowiedź:

To liczby pierwsze mniejsze od 20.

Typowe pułapki

- Liczba:

1

nie jest pierwsza.

- Nie wykreślamy samej liczby pierwszej.
- Łatwo zapomnieć o wykreśleniu wszystkich wielokrotności.
- Wystarczy wykonywać algorytm do:

$\sqrt{n}$

- Każda niewykreślona liczba jest pierwsza.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Znajdź liczby pierwsze mniejsze od 20.

Rozwiązanie:

Po zastosowaniu sita Eratostenesa otrzymujemy:

2, 3, 5, 7, 11, 13, 17, 19

Odpowiedź:

Liczby pierwsze mniejsze od 20:

2, 3, 5, 7, 11, 13, 17, 19

Zadanie 2. Jakie wielokrotności wykreślamy dla liczby 2 przy $n = 20$?

Rozwiązanie:

4, 6, 8, 10, 12, 14, 16, 18, 20

Odpowiedź:

Wielokrotności liczby 2.

Zadanie 3. Prawda czy fałsz?

Liczba 1 jest liczbą pierwszą.

Odpowiedź: Fałsz.

Uzasadnienie:

Liczba pierwsza musi mieć dokładnie dwa dzielniki.

Zadanie 4. Prawda czy fałsz?

W sicie Eratostenesa wykreślamy wielokrotności liczb pierwszych.

Odpowiedź: Prawda.

Uzasadnienie:

To podstawowa idea działania algorytmu.

Zadanie 5. Prawda czy fałsz?

Każda niewykreślona liczba po wykonaniu sita jest liczbą pierwszą.

Odpowiedź: Prawda.

Uzasadnienie:

Wszystkie liczby z dzielnikami zostały wcześniej wykreślone.

Zadanie 6. Do czego używa się sita Eratostenesa?**Odpowiedź:**

Sito Eratostenesa służy do:

- znajdowania liczb pierwszych,
- generowania liczb pierwszych,
- nauki testów pierwszości,
- przygotowania danych do kryptografii.

1.10 Test Lucasa–Lehmera

Definicja

Test Lucasa–Lehmera służy do sprawdzania pierwszości liczb Mersenne’a. Liczby Mersenne’a mają postać:

$$M_p = 2^p - 1$$

gdzie:

$$p$$

jest liczbą pierwszą.

Test jest dużo szybszy od zwykłego trial division dla bardzo dużych liczb Mersenne’a. Był używany na zajęciach razem z liczbami Mersenne’a.

Najważniejsze wzory

Liczba Mersenne’a:

$$M_p = 2^p - 1$$

Ciąg Lucasa–Lehmera:

$$s_0 = 4$$

$$s_{n+1} = s_n^2 - 2$$

Po każdym kroku wykonujemy modulo:

$$M_p$$

Warunek pierwszości:

Po wykonaniu:

$$p - 2$$

iteracji:

$$s_{p-2} \equiv 0 \pmod{M_p}$$

wtedy liczba M_p jest pierwsza.

Własności i zasady

- Test działa tylko dla liczb Mersenne’a.
- Liczba musi mieć postać:

$$M_p = 2^p - 1$$

- Najpierw trzeba sprawdzić, czy:

$$p$$

jest pierwsze.

- Wykonujemy:

$$p - 2$$

iteracji ciągu.

- Po każdym kroku redukujemy modulo:

$$M_p$$

- Jeśli końcowy wynik wynosi:

$$0$$

liczba jest pierwsza.

Metoda krok po kroku

Jak wykonać test Lucasa–Lehmera:

1. Oblicz liczbę Mersenne’a:

$$M_p = 2^p - 1$$

2. Ustaw:

$$s_0 = 4$$

3. Licz kolejne wyrazy:

$$s_{n+1} = s_n^2 - 2$$

4. Po każdym kroku wykonuj:

$$\text{mod } M_p$$

5. Wykonaj:

$$p - 2$$

iteracji.

6. Jeśli:

$$s_{p-2} \equiv 0 \pmod{M_p}$$

liczba jest pierwsza.

Przykład z zajęć

Sprawdź pierwszość liczby:

$$M_5 = 2^5 - 1 = 31$$

Ustawiamy:

$$s_0 = 4$$

Liczba iteracji:

$$p - 2 = 5 - 2 = 3$$

Iteracja 1

$$s_1 = 4^2 - 2 = 14$$

$$14 \pmod{31} = 14$$

Iteracja 2

$$s_2 = 14^2 - 2$$

$$196 - 2 = 194$$

$$194 \pmod{31} = 8$$

Iteracja 3

$$s_3 = 8^2 - 2$$

$$64 - 2 = 62$$

$$62 \pmod{31} = 0$$

Otrzymaliśmy:

$$0$$

Odpowiedź:

jest liczbą pierwszą.

Typowe pułapki

- Test działa tylko dla liczb Mersenne'a.
- Łatwo pomylić liczbę iteracji:

$$p - 2$$

- Po każdym kroku trzeba wykonywać modulo:

$$M_p$$

- Najpierw należy sprawdzić, czy:

$$p$$

jest pierwsze.

- Wynik końcowy musi być dokładnie:

$$0$$

Przykładowe zadania z rozwiązaniem

Zadanie 1. Oblicz liczbę Mersenne'a dla $p = 5$.

Rozwiązanie:

$$M_5 = 2^5 - 1$$

$$M_5 = 32 - 1 = 31$$

Odpowiedź:

$$31$$

Zadanie 2. Ile iteracji wykonujemy dla $p = 7$?

Rozwiązanie:

W teście Lucasa–Lehmera wykonujemy:

$$p - 2$$

iteracji.

$$7 - 2 = 5$$

Odpowiedź:

$$5$$

iteracji.

Zadanie 3. Sprawdź pierwszość liczby M_5 .

Rozwiązanie:

$$M_5 = 31$$

Ustawiamy:

$$s_0 = 4$$

$$s_1 = 4^2 - 2 = 14$$

$$14 \mod 31 = 14$$

$$s_2 = 14^2 - 2$$

$$196 - 2 = 194$$

$$194 \mod 31 = 8$$

$$s_3 = 8^2 - 2$$

$$64 - 2 = 62$$

$$62 \mod 31 = 0$$

Otrzymaliśmy:

$$0$$

Odpowiedź:

$$31$$

jest liczbą pierwszą.

Zadanie 4. Prawda czy fałsz?

Test Lucasa–Lehmera działa dla każdej liczby.

Odpowiedź: Fałsz.

Uzasadnienie:

Test działa tylko dla liczb Mersenne’a.

Zadanie 5. Prawda czy fałsz?

W teście Lucasa–Lehmera po każdym kroku wykonujemy modulo.

Odpowiedź: Prawda.

Uzasadnienie:

Po każdej iteracji redukujemy wynik modulo:

$$M_p$$

Zadanie 6. Prawda czy fałsz?

Jeżeli końcowy wynik testu wynosi:

$$0$$

to liczba Mersenne’a jest pierwsza.

Odpowiedź: Prawda.

Uzasadnienie:

To główny warunek testu Lucasa–Lehmera.

Zadanie 7. Do czego używa się testu Lucasa–Lehmera?

Odpowiedź:

Test Lucasa–Lehmera służy do:

- sprawdzania pierwszości liczb Mersenne’a,
- znajdowania bardzo dużych liczb pierwszych,
- zastosowań kryptograficznych,
- badań matematycznych.

1.11 Liczby Mersenne’a

Definicja

Liczby Mersenne’a to liczby postaci:

$$M_p = 2^p - 1$$

gdzie:

$$p$$

jest liczbą naturalną.

Nazwane zostały od matematyka Marina Mersenne’a.

Nie każda liczba Mersenne’a jest pierwsza, nawet jeśli:

$$p$$

jest liczbą pierwszą.

Liczby Mersenne’a są bardzo ważne w:

- kryptografii,
- testach pierwszości,
- testach Lucasa–Lehmera,
- poszukiwaniu bardzo dużych liczb pierwszych.

Najważniejsze wzory

Definicja liczby Mersenne’a:

$$M_p = 2^p - 1$$

Przykłady:

$$M_2 = 2^2 - 1 = 3$$

$$M_3 = 2^3 - 1 = 7$$

$$M_5 = 2^5 - 1 = 31$$

Jeżeli:

$$M_p$$

jest pierwsza, nazywa się ją:

liczbą pierwszą Mersenne’a

Ważna własność:

Jeżeli:

$$M_p$$

jest pierwsza, to:

$$p$$

musi być liczbą pierwszą.

Własności i zasady

- Liczby Mersenne’a mają postać:

$$2^p - 1$$

- Jeśli:

$$p$$

nie jest pierwsze, to:

$$M_p$$

na pewno nie jest pierwsza.

- Nie każda liczba:

$$M_p$$

jest pierwsza.

- Do badania pierwszości liczb Mersenne’a używa się testu Lucasa–Lehmera.
- Bardzo duże liczby pierwsze odkrywane współcześnie często są liczbami Mersenne’a.

Metoda krok po kroku

Jak sprawdzić liczbę Mersenne’a:

- Wybierz:

$$p$$

- Oblicz:

$$M_p = 2^p - 1$$

- Sprawdź, czy:

$$p$$

jest pierwsze.

4. Wykonaj test pierwszości:

- trial division,
- test Lucasa–Lehmera.

5. Jeśli test zakończy się sukcesem:

$$M_p$$

jest liczbą pierwszą Mersenne’a.

Przykład z zajęć

Oblicz liczbę:

$$M_5$$

Podstawiamy do wzoru:

$$M_p = 2^p - 1$$

$$M_5 = 2^5 - 1$$

$$M_5 = 32 - 1 = 31$$

Sprawdzamy pierwszość:

$$31$$

jest liczbą pierwszą.

Odpowiedź:

$$M_5 = 31$$

jest liczbą pierwszą Mersenne’a.

Przykład liczby złożonej:

$$M_{11} = 2^{11} - 1$$

$$M_{11} = 2047$$

$$2047 = 23 \cdot 89$$

Odpowiedź:

2047

nie jest liczbą pierwszą.

Typowe pułapki

- Nie każda liczba:

$$2^p - 1$$

jest pierwsza.

- To, że:

$$p$$

jest pierwsze, NIE gwarantuje pierwszości:

$$M_p$$

- Liczba:

$$M_{11} = 2047$$

jest klasycznym przykładem liczby złożonej.

- Łatwo pomylić:

$$2^p - 1$$

z:

$$2^{p-1}$$

- W testach często pojawia się pytanie:
„Czy każda liczba Mersenne’a jest pierwsza?”

Przykładowe zadania z rozwiązaniem

Zadanie 1. Oblicz:

$$M_3$$

Rozwiązanie:

$$M_3 = 2^3 - 1$$

$$M_3 = 8 - 1 = 7$$

Odpowiedź:

$$7$$

Zadanie 2. Oblicz:

$$M_5$$

Rozwiązanie:

$$M_5 = 2^5 - 1$$

$$M_5 = 32 - 1 = 31$$

Odpowiedź:

$$31$$

Zadanie 3. Czy liczba M_{11} jest pierwsza?

Rozwiązanie:

$$M_{11} = 2^{11} - 1$$

$$M_{11} = 2047$$

Rozkład:

$$2047 = 23 \cdot 89$$

Odpowiedź:

$$2047$$

nie jest liczbą pierwszą.

Zadanie 4. Prawda czy fałsz?

Jeżeli:

$$M_p$$

jest pierwsza, to:

$$p$$

musi być pierwsze.

Odpowiedź: Prawda.

Uzasadnienie:

To podstawowa własność liczb Mersenne’a.

Zadanie 5. Prawda czy fałsz?

Jeżeli:

$$p$$

jest pierwsze, to:

$$M_p$$

na pewno jest pierwsza.

Odpowiedź: Fałsz.

Uzasadnienie:

Przykład:

$$M_{11} = 2047$$

jest liczbą złożoną.

Zadanie 6. Prawda czy fałsz?

Liczby Mersenne’a mają postać:

$$2^p - 1$$

Odpowiedź: Prawda.

Zadanie 7. Do czego używa się liczb Mersenne’a?

Odpowiedź:

Liczby Mersenne’a są używane m.in. w:

- kryptografii,
- testach pierwszości,
- teście Lucasa–Lehmera,
- poszukiwaniu bardzo dużych liczb pierwszych.

1.12 Generator modulo p

Definicja

Generator modulo p (pierwiastek pierwotny modulo p) to liczba, której kolejne potęgi generują wszystkie możliwe niezerowe reszty modulo p .

Generator jest bardzo ważny w:

- Diffie–Hellmanie,
- ElGamalu,
- logarytmie dyskretnym,
- kryptografii asymetrycznej.

W kryptografii generator oznacza się najczęściej:

$$g$$

Najważniejsze wzory

Dla liczby pierwszej:

$$p$$

generator:

$$g$$

powinien generować wszystkie liczby:

$$1, 2, 3, \dots, p-1$$

przez kolejne potęgowanie modulo p :

$$g^1 \mod p$$

$$g^2 \mod p$$

$$g^3 \mod p$$

$$\dots$$

$$g^{p-1} \mod p$$

Jeżeli wszystkie wyniki są różne i obejmują wszystkie liczby od:

$$1$$

do:

to:

$$p - 1$$

$$g$$

jest generatorem modulo p .

Własności i zasady

- Generator działa modulo liczby pierwszej.
- Generator generuje wszystkie możliwe niezerowe reszty modulo p .
- W Diffie–Hellmanie publicznie podaje się:

$$p$$

oraz:

$$g$$

- Generator NIE może powtarzać wartości przed osiągnięciem:

$$p - 1$$

różnych wyników.

- Nie każda liczba jest generatorem.

Metoda krok po kroku

Jak sprawdzić generator modulo p :

1. Wybierz liczbę:

$$g$$

oraz modulo:

$$p$$

2. Licz kolejne potęgi:

$$g^1, g^2, g^3, \dots$$

modulo:

$$p$$

3. Zapisuj otrzymane reszty.

4. Jeśli przed powtórzeniem pojawiły się wszystkie liczby:

$$1, \dots, p-1$$

to:

$$g$$

jest generatorem.

Przykład z zajęć

Sprawdź, czy:

$$g = 2$$

jest generatorem modulo:

$$p = 5$$

Liczymy kolejne potęgi:

$$2^1 \mod 5 = 2$$

$$2^2 \mod 5 = 4$$

$$2^3 \mod 5 = 8 \mod 5 = 3$$

$$2^4 \mod 5 = 16 \mod 5 = 1$$

Otrzymaliśmy:

$$2, 4, 3, 1$$

czyli wszystkie liczby od:

$$1$$

do:

$$4$$

Odpowiedź:

$$2$$

jest generatorem modulo:

$$5$$

Typowe pułapki

- Nie każda liczba jest generatorem.
- Jeśli wartości zaczną się powtarzać zbyt wcześnie, liczba nie jest generatorem.
- Generator musi wygenerować wszystkie liczby:

$$1, \dots, p-1$$

- Łatwo pomylić zwykłe potęgowanie z potęgowaniem modulo.
- W Diffie–Hellmanie generator jest publiczny.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Sprawdź, czy 2 jest generatorem modulo 5.

Rozwiązanie:

$$2^1 \mod 5 = 2$$

$$2^2 \mod 5 = 4$$

$$2^3 \mod 5 = 3$$

$$2^4 \mod 5 = 1$$

Otrzymaliśmy wszystkie liczby:

$$1, 2, 3, 4$$

Odpowiedź:

$$2$$

jest generatorem modulo:

$$5$$

Zadanie 2. Sprawdź, czy 2 jest generatorem modulo 7.

Rozwiązanie:

$$2^1 \mod 7 = 2$$

$$2^2 \mod 7 = 4$$

$$2^3 \mod 7 = 8 \mod 7 = 1$$

Wartości zaczęły się powtarzać zbyt wcześnie.

Nie otrzymaliśmy wszystkich liczb:

$$1, \dots, 6$$

Odpowiedź:

$$2$$

nie jest generatorem modulo:

$$7$$

Zadanie 3. Prawda czy fałsz?

Generator modulo p generuje wszystkie niezerowe reszty modulo p .

Odpowiedź: Prawda.

Uzasadnienie:

To definicja generatora modulo p .

Zadanie 4. Prawda czy fałsz?

Każda liczba modulo p jest generatorem.

Odpowiedź: Fałsz.

Uzasadnienie:

Niektóre liczby zaczynają powtarzać wyniki zbyt wcześnie.

Zadanie 5. Do czego używa się generatora modulo p ?

Odpowiedź:

Generator modulo p jest używany m.in. w:

- Diffie–Hellmanie,
- ElGamalu,
- logarytmie dyskretnym,
- kryptografii asymetrycznej.

1.13 Logarytm dyskretny

Definicja

Logarytm dyskretny polega na znalezieniu wykładnika:

$$x$$

w równaniu:

$$g^x \equiv y \pmod{p}$$

gdzie:

- g – generator modulo p ,
- p – liczba pierwsza,
- y – znany wynik,
- x – szukany wykładnik.

Problem logarytmu dyskretnego jest podstawą bezpieczeństwa:

- Diffie–Hellmana,
- ElGamala,
- wielu algorytmów kryptografii asymetrycznej.

Najważniejsze wzory

Podstawowe równanie:

$$g^x \equiv y \pmod{p}$$

Szukamy:

$$x$$

Przykład:

$$2^x \equiv 3 \pmod{5}$$

Sprawdzamy kolejne potęgi:

$$2^1 \pmod{5} = 2$$

$$2^2 \pmod{5} = 4$$

$$2^3 \pmod{5} = 8 \pmod{5} = 3$$

czyli:

$$x = 3$$

Własności i zasady

- Logarytm dyskretny jest „odwrotnością” potęgowania modulo.
- Potęgowanie modulo jest szybkie.
- Znalezienie logarytmu dyskretnego dla dużych liczb jest bardzo trudne.
- Na tej trudności opiera się bezpieczeństwo Diffie–Hellmana i ElGamala.
- Generator modulo:

$$g$$

powinien generować wszystkie możliwe reszty modulo p .

Metoda krok po kroku

Jak znaleźć logarytm dyskretny ręcznie:

1. Dane:

$$g^x \equiv y \pmod{p}$$

2. Licz kolejne potęgi:

$$g^1, g^2, g^3, \dots$$

modulo:

$$p$$

3. Sprawdź, kiedy otrzymasz:

$$y$$

4. Numer wykładnika to:

$$x$$

Schemat z zajęć:

$$2^x \equiv 3 \pmod{5}$$

$$2^1 \pmod{5} = 2$$

$$2^2 \bmod 5 = 4$$

$$2^3 \bmod 5 = 3$$

czyli:

$$x = 3$$

Przykład z zajęć

Znajdź:

$$x$$

dla:

$$2^x \equiv 3 \pmod{5}$$

Liczymy kolejne potęgi:

$$2^1 \bmod 5 = 2$$

$$2^2 \bmod 5 = 4$$

$$2^3 \bmod 5 = 8 \bmod 5 = 3$$

Otrzymaliśmy szukaną wartość.

Odpowiedź:

$$x = 3$$

Kolejny przykład:

$$3^x \equiv 4 \pmod{7}$$

Liczymy:

$$3^1 \bmod 7 = 3$$

$$3^2 \bmod 7 = 9 \bmod 7 = 2$$

$$3^3 \bmod 7 = 6$$

$$3^4 \bmod 7 = 18 \bmod 7 = 4$$

Odpowiedź:

$$x = 4$$

Typowe pułapki

- Łatwo pomylić logarytm zwykły z logarytmem dyskretnym.
- Wszystkie obliczenia wykonujemy modulo:

$$p$$

- Generator:

$$g$$

powinien być poprawnym generatorem modulo p .

- Dla dużych liczb problem staje się bardzo trudny obliczeniowo.
- W kryptografii właśnie ta trudność daje bezpieczeństwo.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Znajdź:

$$x$$

dla:

$$2^x \equiv 3 \pmod{5}$$

Rozwiązanie:

$$2^1 \pmod{5} = 2$$

$$2^2 \pmod{5} = 4$$

$$2^3 \pmod{5} = 8 \pmod{5} = 3$$

Odpowiedź:

$$x = 3$$

Zadanie 2. Znajdź:

$$x$$

dla:

$$3^x \equiv 4 \pmod{7}$$

Rozwiązanie:

$$3^1 \pmod{7} = 3$$

$$3^2 \mod 7 = 2$$

$$3^3 \mod 7 = 6$$

$$3^4 \mod 7 = 4$$

Odpowiedź:

$$x = 4$$

Zadanie 3. Znajdź:

$$x$$

dla:

$$5^x \equiv 6 \pmod{7}$$

Rozwiązanie:

$$5^1 \mod 7 = 5$$

$$5^2 \mod 7 = 25 \mod 7 = 4$$

$$5^3 \mod 7 = 20 \mod 7 = 6$$

Odpowiedź:

$$x = 3$$

Zadanie 4. Prawda czy fałsz?

Logarytm dyskretny jest podstawą bezpieczeństwa Diffie–Hellmana.

Odpowiedź: Prawda.

Uzasadnienie:

Bezpieczeństwo Diffie–Hellmana opiera się na trudności rozwiązania problemu logarytmu dyskretnego.

Zadanie 5. Prawda czy fałsz?

Potęgowanie modulo jest trudniejsze niż obliczanie logarytmu dyskretnego.

Odpowiedź: Fałsz.

Uzasadnienie:

Potęgowanie modulo jest szybkie, a logarytm dyskretny dla dużych liczb jest bardzo trudny.

Zadanie 6. Do czego używa się logarytmu dyskretnego?

Odpowiedź:

Problem logarytmu dyskretnego jest używany m.in. w:

- Diffie–Hellmanie,
- ElGamalu,
- kryptografii asymetrycznej,
- wymianie kluczy.

2 Klasyczne szyfry i analiza

2.1 Szyfr Cezara

Definicja

Szyfr Cezara to jeden z najprostszych szyfrów klasycznych. Polega na przesuwaniu każdej litery alfabetu o ustaloną liczbę pozycji. Przykład przesunięcia o 3:

$$A \rightarrow D$$

$$B \rightarrow E$$

$$C \rightarrow F$$

Szyfr Cezara jest szyfrem:

- podstawieniowym,
- symetrycznym,
- monoalfabetycznym.

Najważniejsze wzory

Szyfrowanie:

$$C = (P + k) \mod 26$$

Deszyfrowanie:

$$P = (C - k) \mod 26$$

gdzie:

- P – litera tekstu jawnego,
- C – litera szyfrogramu,
- k – klucz (przesunięcie).

Najczęściej używamy numeracji:

$$A = 0, B = 1, C = 2, \dots, Z = 25$$

Własności i zasady

- Każda litera przesuwana jest o tę samą wartość.
- Kluczem jest liczba przesunięcia:

k

- Operacje wykonujemy modulo:

26

dla alfabetu angielskiego.

- Szyfr Cezara jest bardzo łatwy do złamania.
- Można użyć brute force, ponieważ istnieje tylko:

25

możliwych przesunięć.

Metoda krok po kroku

Jak szyfrować:

1. Zamień litery na liczby.
2. Dodaj przesunięcie:

$+k$

3. Wykonaj modulo:

26

4. Zamień liczby z powrotem na litery.

Jak deszyfrować:

1. Zamień litery na liczby.
2. Odejmij przesunięcie:

$-k$

3. Wykonaj modulo:

26

4. Zamień liczby na litery.

Przykład z zajęć

Zaszyfruj słowo:

HELLO

dla klucza:

$$k = 3$$

Zamiana liter:

$$H = 7, E = 4, L = 11, L = 11, O = 14$$

Dodajemy przesunięcie:

$$7 + 3 = 10$$

$$4 + 3 = 7$$

$$11 + 3 = 14$$

$$11 + 3 = 14$$

$$14 + 3 = 17$$

Zamiana na litery:

$$10 = K$$

$$7 = H$$

$$14 = O$$

$$14 = O$$

$$17 = R$$

Odpowiedź:

HELLO → *KHOOR*

Deszyfrowanie:

KHOOR

dla:

$$k = 3$$

$$K = 10$$

$$10 - 3 = 7 = H$$

Po wykonaniu dla wszystkich liter:

$$KHOOR \rightarrow HELLO$$

Typowe pułapki

- Operacje wykonujemy modulo:

$$26$$

- Łatwo pomylić numerację liter:

$$A = 0$$

a nie:

$$A = 1$$

- Przy deszyfrowaniu odejmujemy klucz.
- Wynik ujemny trzeba sprowadzić modulo:

$$26$$

- Szyfr Cezara NIE jest bezpieczny kryptograficznie.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Zaszzyfruj słowo:

$$CAT$$

dla:

$$k = 3$$

Rozwiązanie:

$$C = 2, A = 0, T = 19$$

Dodajemy przesunięcie:

$$2 + 3 = 5$$

$$0 + 3 = 3$$

$$19 + 3 = 22$$

Zamiana na litery:

$$5 = F, 3 = D, 22 = W$$

Odpowiedź:

$$CAT \rightarrow FDW$$

Zadanie 2. Odszyfruj:

$$KHOO R$$

dla:

$$k = 3$$

Rozwiązanie:

$$K = 10, H = 7, O = 14, O = 14, R = 17$$

Odejmujemy klucz:

$$10 - 3 = 7 = H$$

$$7 - 3 = 4 = E$$

$$14 - 3 = 11 = L$$

$$14 - 3 = 11 = L$$

$$17 - 3 = 14 = O$$

Odpowiedź:

$$HELLO$$

Zadanie 3. Prawda czy fałsz?

Szyfr Cezara jest szyfrem symetrycznym.

Odpowiedź: Prawda.

Uzasadnienie:

Do szyfrowania i deszyfrowania używa się tego samego klucza.

Zadanie 4. Prawda czy fałsz?

Szyfr Cezara jest bezpieczny kryptograficznie.

Odpowiedź: Fałsz.

Uzasadnienie:

Można go łatwo złamać brute force.

Zadanie 5. Ile możliwych kluczy ma szyfr Cezara?

Rozwiązanie:

Dla alfabetu angielskiego:

26

liter.

Pomijamy przesunięcie o:

0

czyli:

25

możliwych kluczy.

Odpowiedź:

25

Zadanie 6. Do jakiego typu szyfrów należy szyfr Cezara?

Odpowiedź:

Szyfr Cezara jest:

- szyfrem podstawieniowym,
- szyfrem symetrycznym,
- szyfrem monoalfabetycznym.

2.2 Szyfr Vigenère'a

Definicja

Szyfr Vigenère'a to klasyczny szyfr polialfabetyczny wykorzystujący słowo-klucz. Każda litera tekstu szyfrowana jest innym przesunięciem zależnym od odpowiedniej litery klucza.

Szyfr Vigenère'a jest dużo trudniejszy do złamania niż szyfr Cezara, ponieważ używa wielu przesunięć.

Na zajęciach pojawiały się:

- szyfrowanie,
- deszyfrowanie,
- analiza IC,
- metoda Kasiski.

Najważniejsze wzory

Szyfrowanie:

$$C_i = (P_i + K_i) \mod 26$$

Deszyfrowanie:

$$P_i = (C_i - K_i) \mod 26$$

gdzie:

- P_i – litera tekstu jawnego,
- C_i – litera szyfrogramu,
- K_i – litera klucza.

Numeracja liter:

$$A = 0, B = 1, C = 2, \dots, Z = 25$$

Własności i zasady

- Klucz jest powtarzany nad tekstem.
- Każda litera może być przesunięta o inną wartość.
- Operacje wykonujemy modulo:

26

- Szyfr jest polialfabetyczny.

- Długość klucza można próbować znaleźć metodą:
 - IC,
 - Kasiski.
- Ten sam znak może być szyfrowany różnie w zależności od pozycji.

Metoda krok po kroku

Jak szyfrować:

1. Zamień litery tekstu i klucza na liczby.
2. Powtarzaj klucz nad tekstem.
3. Dodawaj:

$$P_i + K_i$$

4. Wykonuj modulo:

$$26$$

5. Zamień liczby na litery.

Jak deszyfrować:

1. Zamień litery na liczby.
2. Odejmuj:

$$C_i - K_i$$

3. Wykonuj modulo:

$$26$$

4. Zamień liczby na litery.

Przykład z zajęć

Zaszyfruj:

HELLO

kluczem:

KEY

Powtarzamy klucz:

KEYKE

Zamiana liter:

$$H = 7, E = 4, L = 11, L = 11, O = 14$$

$$K = 10, E = 4, Y = 24, K = 10, E = 4$$

Dodawanie modulo 26:

$$7 + 10 = 17$$

$$4 + 4 = 8$$

$$11 + 24 = 35 \mod 26 = 9$$

$$11 + 10 = 21$$

$$14 + 4 = 18$$

Zamiana na litery:

$$17 = R$$

$$8 = I$$

$$9 = J$$

$$21 = V$$

$$18 = S$$

Odpowiedź:

HELLO → RIJVS

Typowe pułapki

- Klucz trzeba poprawnie powtarzać nad tekstem.
- Operacje wykonujemy modulo:

26

- Łatwo pomylić:

$$A = 0$$

z:

$$A = 1$$

- Przy deszyfrowaniu odejmujemy klucz.
- Wyniki ujemne trzeba sprowadzić modulo:

26

- Ten sam znak może dawać różne wyniki.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Zaszyfruj:

HELLO

kluczem:

KEY

Rozwiązanie:

Powtarzamy klucz:

KEYKE

$$H = 7, E = 4, L = 11, L = 11, O = 14$$

$$K = 10, E = 4, Y = 24, K = 10, E = 4$$

Dodawanie modulo 26:

$$7 + 10 = 17 = R$$

$$4 + 4 = 8 = I$$

$$11 + 24 = 35 \mod 26 = 9 = J$$

$$11 + 10 = 21 = V$$

$$14 + 4 = 18 = S$$

Odpowiedź:

RIJVS

Zadanie 2. Odszyfruj:

RIJVS

kluczem:

KEY

Rozwiązanie:

Powtarzamy klucz:

KEYKE

$$R = 17, I = 8, J = 9, V = 21, S = 18$$

$$K = 10, E = 4, Y = 24, K = 10, E = 4$$

Odejmujemy:

$$17 - 10 = 7 = H$$

$$8 - 4 = 4 = E$$

$$9 - 24 = -15 \mod 26 = 11 = L$$

$$21 - 10 = 11 = L$$

$$18 - 4 = 14 = O$$

Odpowiedź:

HELLO

Zadanie 3. Prawda czy fałsz?

Szyfr Vigenère'a jest szyfrem polialfabetycznym.

Odpowiedź: Prawda.

Uzasadnienie:

Używa wielu różnych przesunięć zależnych od klucza.

Zadanie 4. Prawda czy fałsz?

Ten sam znak zawsze szyfrowany jest tak samo.

Odpowiedź: Fałsz.

Uzasadnienie:

Wynik zależy od aktualnej litery klucza.

Zadanie 5. Prawda czy fałsz?

W szyfrze Vigenère'a używa się modulo:

26

Odpowiedź: Prawda.

Zadanie 6. Jak można analizować szyfr Vigenère'a?

Odpowiedź:

Do analizy używa się m.in.:

- Index of Coincidence,
- metody Kasiski,
- analizy częstotliwości.

Zadanie 7. Do jakiego typu szyfrów należy szyfr Vigenère'a?

Odpowiedź:

Szyfr Vigenère'a jest:

- szyfrem klasycznym,
- szyfrem symetrycznym,
- szyfrem polialfabetycznym.

2.3 Index of Coincidence (IC)

Definicja

Index of Coincidence (IC) to współczynnik używany do analizy tekstu szyfrowanego. Pozwala określić, czy tekst przypomina:

- zwykły język,
- losowy tekst,
- szyfr Vigenère'a.

IC był używany na zajęciach do:

- szacowania długości klucza,
- łamania szyfru Vigenère'a,
- analizy częstotliwości liter.

Najważniejsze wzory

Wzór na IC:

$$IC = \frac{\sum f_i(f_i - 1)}{N(N - 1)}$$

gdzie:

- f_i – liczba wystąpień litery,
- N – długość tekstu.

Typowe wartości:

- język naturalny:

$$IC \approx 0.065$$

- tekst losowy:

$$IC \approx 0.038$$

Własności i zasady

- IC mierzy prawdopodobieństwo trafienia dwóch takich samych liter.
- Im bardziej tekst przypomina naturalny język, tym większy IC.
- Dla szyfru Vigenère'a IC pomaga znaleźć długość klucza.

- Jeśli podzielimy tekst na grupy zgodnie z długością klucza, każda grupa zaczyna przypominać szyfr Cezara.
- Analiza IC była jedną z metod łamania szyfru Vigenère'a na zajęciach.

Metoda krok po kroku

Jak policzyć IC:

1. Policz długość tekstu:

$$N$$

2. Policz wystąpienia każdej litery:

$$f_A, f_B, f_C, \dots$$

3. Oblicz:

$$f_i(f_i - 1)$$

dla każdej litery.

4. Zsumuj wszystkie wartości.

5. Podziel przez:

$$N(N - 1)$$

Interpretacja:

- duży IC $\rightarrow$ tekst przypomina język naturalny,
- mały IC $\rightarrow$ tekst bardziej losowy.

Przykład z zajęć

Rozważmy tekst:

ABABAA

Długość tekstu:

$$N = 6$$

Wystąpienia liter:

$$A = 4$$

$$B = 2$$

Liczymy licznik:

$$4(4 - 1) = 12$$

$$2(2 - 1) = 2$$

$$12 + 2 = 14$$

Liczymy mianownik:

$$6(6 - 1) = 30$$

IC:

$$IC = \frac{14}{30}$$

$$IC \approx 0.467$$

Odpowiedź:

$$IC \approx 0.467$$

Typowe pułapki

- Łatwo pomylić:

$$N$$

z liczbą różnych liter.

- W liczniku używamy:

$$f_i(f_i - 1)$$

a nie:

$$f_i^2$$

- IC nie służy bezpośrednio do odszyfrowania tekstu.
- IC pomaga głównie znaleźć długość klucza.
- Tekst musi być odpowiednio długi, aby IC miał sens.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Oblicz IC dla tekstu:

AAAA

Rozwiązanie:

$$N = 4$$

$$A = 4$$

Licznik:

$$4(4 - 1) = 12$$

Mianownik:

$$4(4 - 1) = 12$$

$$IC = \frac{12}{12} = 1$$

Odpowiedź:

$$IC = 1$$

Zadanie 2. Oblicz IC dla tekstu:

ABABAA

Rozwiązanie:

$$N = 6$$

$$A = 4, B = 2$$

Licznik:

$$4(4 - 1) + 2(2 - 1)$$

$$12 + 2 = 14$$

Mianownik:

$$6(6 - 1) = 30$$

$$IC = \frac{14}{30}$$

$$IC \approx 0.467$$

Odpowiedź:

$$IC \approx 0.467$$

Zadanie 3. Prawda czy fałsz?

IC pomaga analizować szyfr Vigenère'a.

Odpowiedź: Prawda.

Uzasadnienie:

IC służy do szacowania długości klucza.

Zadanie 4. Prawda czy fałsz?

Tekst losowy ma zwykle większy IC niż język naturalny.

Odpowiedź: Fałsz.

Uzasadnienie:

Język naturalny ma większą powtarzalność liter.

Zadanie 5. Jakie typowe IC ma język naturalny?

Odpowiedź:

Około:

$$0.065$$

Zadanie 6. Jakie typowe IC ma tekst losowy?

Odpowiedź:

Około:

$$0.038$$

Zadanie 7. Do czego używa się IC?

Odpowiedź:

IC używa się m.in. do:

- analizy szyfru Vigenère'a,
- szacowania długości klucza,
- analizy częstotliwości,
- rozpoznawania tekstu naturalnego.

2.4 Metoda Kasiski

Definicja

Metoda Kasiski to technika używana do łamania szyfru Vigenère'a. Pozwala oszacować długość klucza poprzez analizę powtarzających się fragmentów szyfrogramu.

Pomysł metody:

jeśli ten sam fragment tekstu został zaszyfrowany tym samym fragmentem klucza, w szyfrogramie pojawiają się powtórzenia.

Odległości między powtórzeniami często są wielokrotnościami długości klucza.

Najważniejsze wzory

Metoda Kasiski nie ma jednego głównego wzoru.

Najważniejsza idea:

$$\gcd(d_1, d_2, d_3, \dots)$$

gdzie:

$$d_i$$

to odległości między powtarzającymi się fragmentami.

Najczęściej szukamy:

$$NWD(d_1, d_2)$$

lub wspólnych dzielników odległości.

Przykład:

$$12, 18, 24$$

Wspólny dzielnik:

$$6$$

Możliwa długość klucza:

$$6$$

Własności i zasady

- Metoda Kasiski działa głównie dla szyfru Vigenère'a.
- Szukamy powtarzających się fragmentów:

$$2 - 4$$

znakowych.

- Mierzmy odległości między powtórzeniami.

- Obliczamy wspólne dzielniki odległości.
- Otrzymana wartość jest kandydatem na długość klucza.
- Metodę często łączy się z:
 - IC,
 - analizą częstotliwości.

Metoda krok po kroku

Jak działa metoda Kasiski:

1. Znajdź powtarzające się fragmenty szyfrogramu.
2. Zapisz ich pozycje.
3. Oblicz odległości między powtórzeniami.
4. Znajdź:

$$NWD$$

lub wspólne dzielniki odległości.

5. Otrzymane wartości traktujemy jako możliwe długości klucza.

Schemat z zajęć:

Powtórzenie:

$$ABC$$

na pozycjach:

$$5$$

oraz:

$$17$$

Odległość:

$$17 - 5 = 12$$

Możliwa długość klucza:

$$2, 3, 4, 6, 12$$

Przykład z zajęć

Rozważmy szyfrogram zawierający fragment:

ABC

który pojawia się na pozycjach:

4, 16, 28

Liczymy odległości:

$$16 - 4 = 12$$

$$28 - 16 = 12$$

Obliczamy wspólne dzielniki:

$$12 = 2 \cdot 6$$

Możliwe długości klucza:

2, 3, 4, 6, 12

Najbardziej prawdopodobna:

6

Odpowiedź:

Długość klucza prawdopodobnie wynosi:

6

Typowe pułapki

- Powtarzające się fragmenty mogą pojawić się przypadkowo.
- Nie każda odległość oznacza dokładną długość klucza.
- Trzeba analizować wspólne dzielniki wielu odległości.
- Metoda działa najlepiej dla dłuższych tekstów.
- Metoda Kasiski nie odszyfrowuje tekstu bezpośrednio.
- Służy głównie do znalezienia długości klucza.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Fragment szyfrogramu powtarza się na pozycjach:

5

oraz:

17

Znajdź odległość.

Rozwiązanie:

$$17 - 5 = 12$$

Odpowiedź:

12

Zadanie 2.

Odległości wynoszą:

12, 18, 24

Znajdź możliwą długość klucza.

Rozwiązanie:

Szukamy wspólnych dzielników.

$$12 = 2^2 \cdot 3$$

$$18 = 2 \cdot 3^2$$

$$24 = 2^3 \cdot 3$$

Wspólne dzielniki:

2, 3, 6

Najbardziej prawdopodobna długość klucza:

6

Odpowiedź:

Możliwa długość klucza:

6

Zadanie 3. Prawda czy fałsz?

Metoda Kasiski służy do analizy szyfru Vigenère'a.

Odpowiedź: Prawda.

Uzasadnienie:

Metoda pozwala oszacować długość klucza szyfru Vigenère'a.

Zadanie 4. Prawda czy fałsz?

Metoda Kasiski bezpośrednio odszyfrowuje tekst.

Odpowiedź: Fałsz.

Uzasadnienie:

Metoda pomaga znaleźć długość klucza, ale nie odszyfrowuje tekstu sama.

Zadanie 5. Prawda czy fałsz?

Powtarzające się fragmenty mogą pojawić się przypadkowo.

Odpowiedź: Prawda.

Uzasadnienie:

Dlatego analizuje się wiele powtórzeń i wiele odległości.

Zadanie 6. Co analizujemy w metodzie Kasiski?

Odpowiedź:

Analizujemy:

- powtarzające się fragmenty szyfrogramu,
- odległości między nimi,
- wspólne dzielniki odległości.

Zadanie 7. Z jakimi metodami często łączy się metodę Kasiski?

Odpowiedź:

Najczęściej z:

- Index of Coincidence,
- analizą częstotliwości,
- szyfrem Vigenère'a.

2.5 ASCII / UTF-8 / reprezentacja tekstu

Definicja

Komputery przechowują tekst jako liczby.

Każdy znak (litera, cyfra, spacja, symbol) ma przypisany kod liczbowy.

Najważniejsze systemy reprezentacji tekstu:

- ASCII,
- UTF-8.

Temat był ważny na zajęciach przy:

- RSA,
- ElGamalu,
- zamianie tekstu na liczby,
- szyfrowaniu plików.

Najważniejsze informacje

ASCII używa:

7

bitów.

Pozwala zapisać:

128

znaków.

Przykłady ASCII:

$A = 65$

$B = 66$

$a = 97$

$0 = 48$

UTF-8 to rozszerzenie ASCII.

UTF-8 używa:

- 1 bajtu dla ASCII,
- większej liczby bajtów dla znaków narodowych.

Przykład:

ą, ć, ł, ź

wymagają więcej niż 1 bajtu.

Własności i zasady

- Komputer przechowuje tekst jako liczby.
- ASCII obsługuje głównie alfabet angielski.
- UTF-8 obsługuje praktycznie wszystkie języki świata.
- UTF-8 jest kompatybilny z ASCII.
- W kryptografii tekst często zamienia się na liczby przed szyfrowaniem.
- W RSA i ElGamalu szyfrujemy liczby, nie litery.

Metoda krok po kroku

Jak zamienić tekst na liczby:

1. Weź znak.
2. Odczytaj jego kod ASCII lub UTF-8.
3. Zamień znak na liczbę.
4. W kryptografii liczby można:
 - szyfrować,
 - łączyć w bloki,
 - haszować.

Schemat z zajęć:

HELLO

$H = 72$

$E = 69$

$L = 76$

$L = 76$

$O = 79$

Przykład z zajęć

Zamień tekst:

ABC

na ASCII.

Kody ASCII:

$A = 65$

$B = 66$

$C = 67$

Odpowiedź:

$ABC \rightarrow 65\ 66\ 67$

Przykład UTF-8:

Znak:

ą

nie istnieje w podstawowym ASCII.

UTF-8 zapisuje go przy użyciu większej liczby bajtów.

Przykład kryptograficzny:

HELLO

72 69 76 76 79

Takie liczby można później szyfrować RSA lub ElGamalem.

Typowe pułapki

- ASCII i UTF-8 to nie to samo.
- ASCII nie obsługuje polskich znaków.
- UTF-8 może używać wielu bajtów dla jednego znaku.
- W kryptografii szyfrujemy liczby, nie litery.
- Łatwo pomylić:

ASCII

z:

Przykładowe zadania z rozwiązaniem

Zadanie 1. Zamień na ASCII:

ABC

Rozwiązanie:

$$A = 65$$

$$B = 66$$

$$C = 67$$

Odpowiedź:

65 66 67

Zadanie 2. Zamień na ASCII:

HELLO

Rozwiązanie:

$$H = 72$$

$$E = 69$$

$$L = 76$$

$$L = 76$$

$$O = 79$$

Odpowiedź:

72 69 76 76 79

Zadanie 3. Prawda czy fałsz?

ASCII obsługuje polskie znaki.

Odpowiedź: Fałsz.

Uzasadnienie:

Podstawowe ASCII zawiera głównie znaki alfabetu angielskiego.

Zadanie 4. Prawda czy fałsz?

UTF-8 jest kompatybilny z ASCII.

Odpowiedź: Prawda.

Uzasadnienie:

Znaki ASCII mają takie same kody w UTF-8.

Zadanie 5. Prawda czy fałsz?

W RSA szyfrujemy bezpośrednio litery.

Odpowiedź: Fałsz.

Uzasadnienie:

Najpierw tekst zamienia się na liczby.

Zadanie 6. Ile znaków obsługuje podstawowe ASCII?

Odpowiedź:

128

znaków.

Zadanie 7. Do czego używa się ASCII i UTF-8?

Odpowiedź:

ASCII i UTF-8 służą do:

- reprezentacji tekstu,
- przechowywania znaków,
- przesyłania tekstu,
- kryptografii,
- kodowania danych.

3 Kryptografia asymetryczna

3.1 Diffie–Hellman

Definicja

Algorytm Diffie–Hellmana służy do bezpiecznej wymiany klucza przez niezabezpieczony kanał komunikacyjny.

Dwie osoby mogą wspólnie wygenerować ten sam sekret, mimo że podsłuchujący zna wszystkie publiczne dane.

Bezpieczeństwo Diffie–Hellmana opiera się na trudności problemu logarytmu dyskretnego.

Algorytm jest podstawą wielu nowoczesnych protokołów kryptograficznych.

Najważniejsze wzory

Publiczne dane:

$$p$$

– liczba pierwsza

$$g$$

– generator modulo p

Alice wybiera sekret:

$$a$$

Bob wybiera sekret:

$$b$$

Klucze publiczne:

$$A = g^a \mod p$$

$$B = g^b \mod p$$

Wspólny sekret:

$$K = B^a \mod p$$

$$K = A^b \mod p$$

Obie strony otrzymują ten sam wynik.

Własności i zasady

- Diffie–Hellman służy do wymiany klucza, a nie bezpośredniego szyfrowania wiadomości.
- Publicznie znane są:

$$p, g, A, B$$

- Tajne pozostają:

$$a, b$$

- Obie strony obliczają ten sam wspólny sekret.
- Bezpieczeństwo opiera się na problemie logarytmu dyskretnego.
- Podśluchujący zna dane publiczne, ale nie zna sekretów.

Metoda krok po kroku

Jak działa Diffie–Hellman:

1. Ustalamy publicznie:

$$p, g$$

2. Alice wybiera sekret:

$$a$$

3. Bob wybiera sekret:

$$b$$

4. Alice oblicza:

$$A = g^a \mod p$$

i wysyła:

$$A$$

5. Bob oblicza:

$$B = g^b \mod p$$

i wysyła:

$$B$$

6. Alice liczy:

$$K = B^a \mod p$$

7. Bob liczy:

$$K = A^b \mod p$$

8. Obie strony otrzymują ten sam klucz:

$$K$$

Przykład z zajęć

Dane publiczne:

$$p = 23$$

$$g = 5$$

Alice wybiera:

$$a = 6$$

Bob wybiera:

$$b = 15$$

Alice oblicza:

$$A = 5^6 \mod 23$$

$$5^6 = 15625$$

$$15625 \mod 23 = 8$$

Bob oblicza:

$$B = 5^{15} \mod 23$$

$$B = 19$$

Wymiana kluczy publicznych:

$$A = 8$$

$$B = 19$$

Alice liczy:

$$K = 19^6 \mod 23$$

$$K = 2$$

Bob liczy:

$$K = 8^{15} \mod 23$$

$$K = 2$$

Odpowiedź:

Wspólny sekret:

$$K = 2$$

Typowe pułapki

- Diffie–Hellman NIE szyfruje wiadomości.
- Służy tylko do wymiany wspólnego klucza.
- Łatwo pomylić:

$$A, B$$

z sekretami:

$$a, b$$

- Sekrety:

$$a, b$$

nigdy nie są wysyłane.

- Wszystkie obliczenia wykonujemy modulo:

$$p$$

- Bezpieczeństwo opiera się na logarytmie dyskretnym.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Dane:

$$p = 11, g = 2$$

Alice wybiera:

$$a = 3$$

Oblicz klucz publiczny Alice.

Rozwiązanie:

$$A = 2^3 \mod 11$$

$$8 \mod 11 = 8$$

Odpowiedź:

$$A = 8$$

Zadanie 2.

Dane:

$$p = 11, g = 2$$

Bob wybiera:

$$b = 5$$

Oblicz klucz publiczny Bob.

Rozwiązanie:

$$B = 2^5 \mod 11$$

$$32 \mod 11 = 10$$

Odpowiedź:

$$B = 10$$

Zadanie 3.

Alice otrzymała:

$$B = 10$$

oraz:

$$a = 3$$

Oblicz wspólny sekret.

Rozwiązanie:

$$K = 10^3 \mod 11$$

$$1000 \mod 11 = 10$$

Odpowiedź:

$$K = 10$$

Zadanie 4. Prawda czy fałsz?

Diffie–Hellman służy do wymiany klucza.

Odpowiedź: Prawda.

Uzasadnienie:

Algorytm pozwala uzgodnić wspólny sekret.

Zadanie 5. Prawda czy fałsz?

Sekrety:

$$a, b$$

są publiczne.

Odpowiedź: Fałsz.

Uzasadnienie:

Sekrety nigdy nie są przesyłane.

Zadanie 6. Prawda czy fałsz?

Bezpieczeństwo Diffie–Hellmana opiera się na logarytmie dyskretnym.

Odpowiedź: Prawda.

Zadanie 7. Jakie dane są publiczne w Diffie–Hellmanie?

Odpowiedź:

Publiczne są:

- p ,
- g ,
- A ,
- B .

Tajne pozostają:

$$a, b$$

3.2 Man In The Middle Attack (MITM)

Definicja

Man In The Middle Attack (MITM) to atak, w którym napastnik przechwytuje komunikację pomiędzy dwiema stronami.

Atakujący podszywa się pod każdą ze stron osobno, dzięki czemu może:

- podsłuchiwać wiadomości,
- zmieniać dane,
- przekazywać fałszywe informacje.

Na zajęciach MITM pojawiał się głównie przy:

- Diffie–Hellmanie,
- wymianie kluczy,
- braku uwierzytelniania.

Najważniejsze informacje

W klasycznym MITM dla Diffie–Hellmana:

$$Alice \leftrightarrow Eve \leftrightarrow Bob$$

zamiast:

$$Alice \leftrightarrow Bob$$

Atakujący:

$$Eve$$

tworzy dwa różne sekrety:

$$K_{AE}$$

oraz:

$$K_{EB}$$

Alice myśli, że rozmawia z Bobem.

Bob myśli, że rozmawia z Alice.

W rzeczywistości obie strony komunikują się z Eve.

Własności i zasady

- Diffie–Hellman bez uwierzytelniania jest podatny na MITM.

- Atakujący przechwytuje klucze publiczne.
- Eve może:
 - odszyfrować wiadomość,
 - zmienić wiadomość,
 - ponownie zaszyfrować wiadomość.
- Obie strony mogą nie zauważyć ataku.
- Przed MITM chronią:
 - podpisy cyfrowe,
 - certyfikaty,
 - uwierzytelnianie.

Jak działa atak MITM

Schemat ataku:

1. Alice wysyła klucz publiczny do Boba.
2. Eve przechwytuje wiadomość.
3. Eve wysyła Bobowi własny klucz publiczny.
4. Bob odpowiada swoim kluczem publicznym.
5. Eve przechwytuje odpowiedź.
6. Eve wysyła Alice własny klucz publiczny.
7. Powstają dwa różne sekrety:

$$K_{AE}$$

oraz:

$$K_{EB}$$

8. Eve może czytać i modyfikować komunikację.

Najważniejsza idea:

Brak uwierzytelniania kluczy publicznych.

Przykład z zajęć

Alice chce wykonać Diffie–Hellmana z Bobem.

Alice wysyła:

$$A = g^a \mod p$$

Eve przechwytuje:

$$A$$

i zamiast niego wysyła Bobowi:

$$E_1 = g^{e_1} \mod p$$

Bob odpowiada:

$$B = g^b \mod p$$

Eve przechwytuje:

$$B$$

i wysyła Alice:

$$E_2 = g^{e_2} \mod p$$

Powstają dwa różne sekrety:

$$K_{AE}$$

$$K_{EB}$$

Eve może teraz:

- odszyfrować wiadomość od Alice,
- przeczytać ją,
- zmodyfikować,
- zaszyfrować ponownie dla Boba.

Typowe pułapki

- Diffie–Hellman sam z siebie NIE uwierzytelnia stron.
- Łatwo pomylić:

podśluch

z:

MITM

MITM aktywnie modyfikuje komunikację.

- Eve tworzy dwa różne sekrety.
- Samo szyfrowanie nie chroni przed MITM.
- Potrzebne są:
 - certyfikaty,
 - podpis cyfrowy,
 - uwierzytelnianie.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Na czym polega atak MITM?

Odpowiedź:

Atakujący przechwytuje komunikację pomiędzy dwiema stronami i podszywa się pod każdą z nich osobno.

Zadanie 2. Prawda czy fałsz?

Diffie–Hellman bez uwierzytelniania jest podatny na MITM.

Odpowiedź: Prawda.

Uzasadnienie:

Atakujący może podmienić klucze publiczne.

Zadanie 3. Prawda czy fałsz?

MITM to tylko bierny podsłuch.

Odpowiedź: Fałsz.

Uzasadnienie:

MITM aktywnie zmienia komunikację.

Zadanie 4.

Jakie dwa sekrety tworzy Eve podczas MITM?

Odpowiedź:

K_{AE}

oraz:

K_{EB}

Zadanie 5. Prawda czy fałsz?

Podpis cyfrowy może chronić przed MITM.

Odpowiedź: Prawda.

Uzasadnienie:

Pozwala uwierzytelnić nadawcę i klucz publiczny.

Zadanie 6.

Co może zrobić atakujący MITM?

Odpowiedź:

Może:

- podsłuchiwać dane,
- zmieniać wiadomości,
- przekazywać fałszywe informacje,
- odszyfrowywać komunikację.

Zadanie 7.

Jak można chronić się przed MITM?

Odpowiedź:

Najczęściej używa się:

- podpisów cyfrowych,
- certyfikatów,
- TLS,
- uwierzytelniania stron.

3.3 RSA

Definicja

RSA to jeden z najważniejszych algorytmów kryptografii asymetrycznej. W RSA używa się dwóch kluczy:

- klucza publicznego,
- klucza prywatnego.

Klucz publiczny służy do szyfrowania.

Klucz prywatny służy do odszyfrowania wiadomości.

Bezpieczeństwo RSA opiera się na trudności rozkładu dużych liczb na czynniki pierwsze.

Najważniejsze wzory

Wybieramy liczby pierwsze:

$$p, q$$

Obliczamy:

$$n = pq$$

Funkcja Eulera:

$$\varphi(n) = (p - 1)(q - 1)$$

Wybieramy:

$$e$$

takie, że:

$$\text{NWD}(e, \varphi(n)) = 1$$

Obliczamy:

$$d = e^{-1} \pmod{\varphi(n)}$$

czyli:

$$ed \equiv 1 \pmod{\varphi(n)}$$

Klucz publiczny:

$$(e, n)$$

Klucz prywatny:

$$(d, n)$$

Szyfrowanie:

$$C = M^e \mod n$$

Deszyfrowanie:

$$M = C^d \mod n$$

Własności i zasady

- RSA jest szyfrem asymetrycznym.
- Klucz publiczny można udostępniać wszystkim.
- Klucz prywatny musi pozostać tajny.
- Wiadomość musi spełniać:

$$M < n$$

- RSA działa dzięki arytmetyce modularnej oraz funkcji Eulera.
- W praktyce RSA używa bardzo dużych liczb pierwszych.

Metoda krok po kroku

Generowanie kluczy RSA:

1. Wybierz liczby pierwsze:

$$p, q$$

2. Oblicz:

$$n = pq$$

3. Oblicz:

$$\varphi(n) = (p - 1)(q - 1)$$

4. Wybierz:

$$e$$

takie, że:

$$\text{NWD}(e, \varphi(n)) = 1$$

5. Oblicz:

$$d = e^{-1} \mod \varphi(n)$$

6. Utwórz:

- klucz publiczny:

$$(e, n)$$

- klucz prywatny:

$$(d, n)$$

Szyfrowanie:

$$C = M^e \mod n$$

Deszyfrowanie:

$$M = C^d \mod n$$

Przykład z zajęć

Wybieramy:

$$p = 3, q = 11$$

Obliczamy:

$$n = 3 \cdot 11 = 33$$

Funkcja Eulera:

$$\varphi(33) = (3 - 1)(11 - 1)$$

$$\varphi(33) = 2 \cdot 10 = 20$$

Wybieramy:

$$e = 3$$

Sprawdzamy:

$$NWD(3, 20) = 1$$

Szukamy:

$$d$$

takiego, że:

$$3d \equiv 1 \pmod{20}$$

$$d = 7$$

ponieważ:

$$3 \cdot 7 = 21 \equiv 1 \pmod{20}$$

Klucz publiczny:

$$(3, 33)$$

Klucz prywatny:

$$(7, 33)$$

Szyfrujemy:

$$M = 4$$

$$C = 4^3 \pmod{33}$$

$$64 \pmod{33} = 31$$

Deszyfrowanie:

$$M = 31^7 \pmod{33}$$

$$M = 4$$

Odpowiedź:

Wiadomość została poprawnie odszyfrowana.

Typowe pułapki

- Liczby:

$$p, q$$

muszą być pierwsze.

- Wiadomość musi spełniać:

$$M < n$$

- Łatwo pomylić:

$$e$$

oraz:

$$d$$

- Trzeba sprawdzić:

$$NWD(e, \varphi(n)) = 1$$

- Wszystkie działania wykonujemy modulo:

$$n$$

- W praktyce używa się szybkiego potęgowania modularnego.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Dane:

$$p = 5, q = 11$$

Oblicz:

$$n$$

Rozwiązanie:

$$n = 5 \cdot 11 = 55$$

Odpowiedź:

$$n = 55$$

Zadanie 2.

Oblicz:

$$\varphi(55)$$

Rozwiązanie:

$$\varphi(55) = (5 - 1)(11 - 1)$$

$$4 \cdot 10 = 40$$

Odpowiedź:

$$\varphi(55) = 40$$

Zadanie 3.

Czy:

$$e = 3$$

może być użyte dla:

$$\varphi(n) = 40$$

Rozwiązanie:

$$NWD(3, 40) = 1$$

Odpowiedź:

Tak.

Zadanie 4.

Znajdź:

$$d$$

dla:

$$e = 3, \varphi(n) = 20$$

Rozwiązanie:

Szukamy:

$$3d \equiv 1 \pmod{20}$$

$$3 \cdot 7 = 21$$

$$21 \equiv 1 \pmod{20}$$

Odpowiedź:

$$d = 7$$

Zadanie 5.

Zaszyfruj:

$$M = 4$$

dla:

$$e = 3, n = 33$$

Rozwiązanie:

$$C = 4^3 \mod 33$$

$$64 \mod 33 = 31$$

Odpowiedź:

$$C = 31$$

Zadanie 6. Prawda czy fałsz?

RSA jest szyfrem asymetrycznym.

Odpowiedź: Prawda.

Uzasadnienie:

Używa klucza publicznego i prywatnego.

Zadanie 7. Prawda czy fałsz?

Klucz prywatny można publikować.

Odpowiedź: Fałsz.

Uzasadnienie:

Klucz prywatny musi pozostać tajny.

Zadanie 8. Na czym opiera się bezpieczeństwo RSA?

Odpowiedź:

Na trudności rozkładu dużych liczb na czynniki pierwsze.

3.4 Zamiana tekstu na liczby / bloki bajtowe

Definicja

W kryptografii komputer nie szyfruje bezpośrednio liter, lecz liczby. Dlatego tekst trzeba najpierw zamienić na reprezentację liczbową, najczęściej:

- ASCII,
- UTF-8,
- bloki bajtowe.

Na zajęciach temat pojawiał się głównie przy:

- RSA,
- ElGamalu,
- szyfrowaniu plików,
- OpenPGP.

Najważniejsze informacje

Przykładowe kody ASCII:

$$A = 65$$

$$B = 66$$

$$H = 72$$

Tekst:

HELLO

można zapisać jako:

72 69 76 76 79

Blok bajtowy:

ciąg kilku znaków zapisanych jako jedna większa liczba.

Przykład:

HI

$$H = 72$$

$$I = 73$$

Blok:

72073

lub zapis binarny/bajtowy.

Własności i zasady

- RSA i ElGamal szyfrują liczby.
- Tekst musi zostać zamieniony na liczby przed szyfrowaniem.
- Wiadomości często dzieli się na bloki.
- Blok musi spełniać:

$$M < n$$

dla RSA.

- UTF-8 może używać wielu bajtów dla jednego znaku.
- W praktyce używa się bloków bajtowych zamiast pojedynczych znaków.

Metoda krok po kroku

Jak zamienić tekst na liczby:

1. Weź tekst.
2. Zamień każdy znak na kod ASCII/UTF-8.
3. Połącz liczby w bloki.
4. Użyj bloków jako wiadomości:

$$M$$

w RSA lub ElGamalu.

Schemat z zajęć:

HELLO

72 69 76 76 79

Możliwe bloki:

7269

7679

Przykład z zajęć

Zamień:

ABC

na liczby ASCII.

$A = 65$

$B = 66$

$C = 67$

Odpowiedź:

65 66 67

Przykład bloków:

HELLO

72 69 76 76 79

Możliwe bloki:

7269

7679

Takie liczby można szyfrować RSA lub ElGamalem.

Typowe pułapki

- RSA szyfruje liczby, nie litery.
- Blok nie może być większy niż:

n

- Łatwo pomylić:

ASCII

z:

wartością binarną

- UTF-8 i ASCII nie są tym samym.
- W praktyce wiadomości dzieli się na wiele bloków.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Zamień na ASCII:

ABC

Rozwiązanie:

$A = 65$

$B = 66$

$C = 67$

Odpowiedź:

65 66 67

Zadanie 2. Zamień na ASCII:

HELLO

Rozwiązanie:

$H = 72$

$E = 69$

$L = 76$

$L = 76$

$O = 79$

Odpowiedź:

72 69 76 76 79

Zadanie 3.

Utwórz blok bajtowy z:

$$HI$$

Rozwiązanie:

$$H = 72$$

$$I = 73$$

Łączymy liczby:

$$72073$$

Odpowiedź:

$$72073$$
Zadanie 4. Prawda czy fałsz?

RSA szyfruje bezpośrednio litery.

Odpowiedź: Fałsz.

Uzasadnienie:

Najpierw tekst zamienia się na liczby.

Zadanie 5. Prawda czy fałsz?

Blok RSA musi być mniejszy niż:

$$n$$

Odpowiedź: Prawda.

Uzasadnienie:

W przeciwnym razie szyfrowanie RSA nie działa poprawnie.

Zadanie 6. Prawda czy fałsz?

UTF-8 może używać wielu bajtów dla jednego znaku.

Odpowiedź: Prawda.

Zadanie 7. Dlaczego używa się bloków bajtowych?

Odpowiedź:

Bloki pozwalają:

- szyfrować większe fragmenty tekstu,
- efektywniej wykorzystywać RSA,
- szyfrować pliki i wiadomości.

3.5 ElGamal

Definicja

ElGamal to algorytm kryptografii asymetrycznej oparty na problemie logarytmu dyskretnego.

Wykorzystuje:

- liczbę pierwszą:

$$p$$

- generator:

$$g$$

- klucz prywatny,
- klucz publiczny.

ElGamal był na zajęciach używany do:

- szyfrowania wiadomości,
- wymiany danych,
- pokazania known plaintext attack.

Najważniejsze wzory

Dane publiczne:

$$p, g$$

Klucz prywatny:

$$x$$

Klucz publiczny:

$$y = g^x \mod p$$

Szyfrowanie wiadomości:

$$m$$

Losujemy:

$$k$$

Obliczamy:

$$c_1 = g^k \mod p$$

$$c_2 = m \cdot y^k \mod p$$

Szyfrogram:

$$(c_1, c_2)$$

Deszyfrowanie:

$$m = c_2 \cdot (c_1^x)^{-1} \mod p$$

Własności i zasady

- ElGamal jest szyfrem asymetrycznym.
- Bezpieczeństwo opiera się na logarytmie dyskretnym.
- Każde szyfrowanie używa losowego:

$$k$$

- Ta sama wiadomość może dawać różne szyfrogramy.
- Wiadomość musi spełniać:

$$m < p$$

- ElGamal używa mnożenia modulo.

Metoda krok po kroku

Generowanie kluczy:

1. Wybierz:

$$p, g$$

2. Wybierz sekret:

$$x$$

3. Oblicz klucz publiczny:

$$y = g^x \mod p$$

Szyfrowanie:

1. Weź wiadomość:

$$m$$

2. Wylosuj:

$$k$$

3. Oblicz:

$$c_1 = g^k \mod p$$

4. Oblicz:

$$c_2 = m \cdot y^k \mod p$$

5. Wyślij:

$$(c_1, c_2)$$

Deszyfrowanie:

1. Oblicz:

$$c_1^x \mod p$$

2. Znajdź odwrotność modularną.

3. Oblicz:

$$m = c_2 \cdot (c_1^x)^{-1} \mod p$$

Przykład z zajęć

Dane publiczne:

$$p = 23$$

$$g = 5$$

Klucz prywatny:

$$x = 6$$

Klucz publiczny:

$$y = 5^6 \mod 23$$

$$y = 8$$

Szyfrujemy wiadomość:

$$m = 10$$

Losujemy:

$$k = 3$$

Obliczamy:

$$c_1 = 5^3 \mod 23$$

$$125 \mod 23 = 10$$

Obliczamy:

$$c_2 = 10 \cdot 8^3 \mod 23$$

$$8^3 = 512$$

$$512 \mod 23 = 6$$

$$10 \cdot 6 = 60$$

$$60 \mod 23 = 14$$

Szyfrogram:

$$(10, 14)$$

Deszyfrowanie:

$$m = 14 \cdot (10^6)^{-1} \mod 23$$

Po obliczeniach:

$$m = 10$$

Odpowiedź:

Wiadomość została poprawnie odszyfrowana.

Typowe pułapki

- Wiadomość musi spełniać:

$$m < p$$

- Nie wolno używać tego samego:

$$k$$

wiele razy.

- Łatwo pomylić:

$$x$$

z:

$$k$$

- Wszystkie działania wykonujemy modulo:

$$p$$

- Do deszyfrowania potrzebna jest odwrotność modularna.
- ElGamal jest podatny na known plaintext attack przy złym użyciu.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Dane:

$$p = 23, \quad g = 5, \quad x = 6$$

Oblicz klucz publiczny.

Rozwiązanie:

$$y = 5^6 \mod 23$$

$$15625 \mod 23 = 8$$

Odpowiedź:

$$y = 8$$

Zadanie 2.

Dane:

$$g = 5, k = 3, p = 23$$

Oblicz:

$$c_1$$

Rozwiązanie:

$$c_1 = 5^3 \mod 23$$

$$125 \mod 23 = 10$$

Odpowiedź:

$$c_1 = 10$$

Zadanie 3.

Dane:

$$m = 10, y = 8, k = 3, p = 23$$

Oblicz:

$$c_2$$

Rozwiązanie:

$$8^3 = 512$$

$$512 \mod 23 = 6$$

$$c_2 = 10 \cdot 6 \mod 23$$

$$60 \mod 23 = 14$$

Odpowiedź:

$$c_2 = 14$$

Zadanie 4. Prawda czy fałsz?

ElGamal jest szyfrem asymetrycznym.

Odpowiedź: Prawda.

Uzasadnienie:

Używa klucza publicznego i prywatnego.

Zadanie 5. Prawda czy fałsz?

Ta sama wiadomość zawsze daje ten sam szyfrogram.

Odpowiedź: Fałsz.

Uzasadnienie:

Losowane jest:

k

więc szyfrogram może być różny.

Zadanie 6. Prawda czy fałsz?

W ElGamalu wiadomość musi być mniejsza niż:

p

Odpowiedź: Prawda.

Zadanie 7. Na czym opiera się bezpieczeństwo ElGamala?

Odpowiedź:

Na trudności problemu logarytmu dyskretnego.

3.6 Klucz publiczny vs klucz prywatny

Definicja

W kryptografii asymetrycznej używa się dwóch różnych kluczy:

- klucza publicznego,
- klucza prywatnego.

Klucz publiczny można udostępniać wszystkim.

Klucz prywatny musi pozostać tajny.

Taki mechanizm jest używany m.in. w:

- RSA,
- ElGamalu,
- podpisie cyfrowym,
- OpenPGP.

Najważniejsze informacje

Klucz publiczny:

Public Key

służy najczęściej do:

- szyfrowania,
- weryfikacji podpisu cyfrowego.

Klucz prywatny:

Private Key

służy najczęściej do:

- odszyfrowania wiadomości,
- tworzenia podpisu cyfrowego.

Przykład RSA:

Klucz publiczny:

(e, n)

Klucz prywatny:

(d, n)

Własności i zasady

- Klucz publiczny można bezpiecznie publikować.
- Klucz prywatny musi być tajny.
- Wiadomość zaszyfrowana kluczem publicznym może zostać odszyfrowana tylko kluczem prywatnym.
- Podpis cyfrowy tworzy się kluczem prywatnym.
- Weryfikacja podpisu odbywa się kluczem publicznym.
- Kryptografia asymetryczna jest wolniejsza od symetrycznej.

Jak działa para kluczy

Szyfrowanie wiadomości:

1. Bob generuje:
 - klucz publiczny,
 - klucz prywatny.
2. Bob publikuje klucz publiczny.
3. Alice szyfruje wiadomość kluczem publicznym Bob'a.
4. Bob odszyfrowuje wiadomość swoim kluczem prywatnym.

Podpis cyfrowy:

1. Alice podpisuje dane kluczem prywatnym.
2. Bob używa klucza publicznego Alice do weryfikacji podpisu.

Przykład z zajęć

RSA:

Klucz publiczny:

$$(e, n) = (3, 33)$$

Klucz prywatny:

$$(d, n) = (7, 33)$$

Alice chce wysłać wiadomość do Bob'a.

Alice używa klucza publicznego:

$(3, 33)$

do szyfrowania.

Bob używa klucza prywatnego:

$(7, 33)$

do odszyfrowania wiadomości.

Podpis cyfrowy działa odwrotnie:

- podpis $\rightarrow$ klucz prywatny,
- verify $\rightarrow$ klucz publiczny.

Typowe pułapki

- Klucz publiczny NIE służy do odszyfrowywania wiadomości.
- Klucz prywatny NIE może zostać ujawniony.
- Łatwo pomylić:
 - szyfrowanie,
 - podpis cyfrowy.
- Podpis cyfrowy działa odwrotnie niż szyfrowanie.
- Kryptografia asymetryczna jest wolniejsza od symetrycznej.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Który klucz można publikować?

Odpowiedź:

Klucz publiczny.

Zadanie 2.

Który klucz musi pozostać tajny?

Odpowiedź:

Klucz prywatny.

Zadanie 3. Prawda czy fałsz?

Wiadomość zaszyfrowana kluczem publicznym może zostać odszyfrowana kluczem prywatnym.

Odpowiedź: Prawda.

Uzasadnienie:

Tak działa kryptografia asymetryczna.

Zadanie 4. Prawda czy fałsz?

Klucz prywatny można wysłać przez Internet wszystkim użytkownikom.

Odpowiedź: Fałsz.

Uzasadnienie:

Klucz prywatny musi być tajny.

Zadanie 5.

Którego klucza używa się do podpisu cyfrowego?

Odpowiedź:

Klucza prywatnego.

Zadanie 6.

Którego klucza używa się do verify podpisu?

Odpowiedź:

Klucza publicznego.

Zadanie 7. Prawda czy fałsz?

RSA jest przykładem kryptografii asymetrycznej.

Odpowiedź: Prawda.

Uzasadnienie:

RSA używa pary kluczy: publicznego i prywatnego.

Zadanie 8.

Do czego służy klucz publiczny?

Odpowiedź:

Najczęściej do:

- szyfrowania,
- verify podpisu cyfrowego.

Zadanie 9.

Do czego służy klucz prywatny?

Odpowiedź:

Najczęściej do:

- odszyfrowywania,
- podpisu cyfrowego.

3.7 Szyfrowanie asymetryczne vs symetryczne

Definicja

Istnieją dwa główne typy kryptografii:

- kryptografia symetryczna,
- kryptografia asymetryczna.

Kryptografia symetryczna używa jednego wspólnego klucza.

Kryptografia asymetryczna używa dwóch różnych kluczy:

- publicznego,
- prywatnego.

Na zajęciach porównywaliśmy oba podejścia przy:

- RSA,
- Diffie–Hellmanie,
- OpenPGP,
- podpisie cyfrowym.

Najważniejsze informacje

Kryptografia symetryczna:

$$K_{enc} = K_{dec}$$

Ten sam klucz służy do:

- szyfrowania,
- odszyfrowania.

Kryptografia asymetryczna:

$$K_{public} \neq K_{private}$$

Klucz publiczny:

- szyfruje,
- verify podpis.

Klucz prywatny:

- odszyfrowuje,
- tworzy podpis.

Własności i zasady

Kryptografia symetryczna:

- bardzo szybka,
- dobra do dużych plików,
- problemem jest bezpieczne przekazanie klucza.

Kryptografia asymetryczna:

- wolniejsza,
- używa dwóch kluczy,
- rozwiązuje problem wymiany kluczy,
- pozwala tworzyć podpis cyfrowy.

W praktyce często używa się obu metod jednocześnie.

Jak działają oba typy kryptografii

Szyfrowanie symetryczne:

1. Alice i Bob znają wspólny klucz.
2. Alice szyfruje wiadomość tym kluczem.
3. Bob używa tego samego klucza do odszyfrowania.

Szyfrowanie asymetryczne:

1. Bob tworzy:
 - klucz publiczny,
 - klucz prywatny.
2. Bob publikuje klucz publiczny.
3. Alice szyfruje wiadomość kluczem publicznym Bob'a.
4. Bob odszyfrowuje wiadomość kluczem prywatnym.

Przykład z zajęć

Kryptografia symetryczna:

Alice i Bob znają wspólny klucz:

$$K = 12345$$

Alice szyfruje wiadomość.

Bob używa tego samego klucza do odszyfrowania.

Kryptografia asymetryczna:

Bob generuje RSA:

$$(e, n)$$

oraz:

$$(d, n)$$

Alice używa:

$$(e, n)$$

do szyfrowania.

Bob używa:

$$(d, n)$$

do odszyfrowania.

OpenPGP z zajęć:

- RSA → wymiana klucza,
- AES/symetryczny → szyfrowanie pliku.

Typowe pułapki

- RSA NIE służy do szyfrowania dużych plików bezpośrednio.
- Kryptografia asymetryczna jest dużo wolniejsza.
- W symetrycznej oba urządzenia muszą znać ten sam klucz.
- Klucz prywatny musi pozostać tajny.
- Łatwo pomylić:
 - szyfrowanie,
 - podpis cyfrowy.
- W praktyce systemy używają obu typów jednocześnie.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Ile kluczy używa kryptografia symetryczna?

Odpowiedź:

Jednego wspólnego klucza.

Zadanie 2.

Ile kluczy używa kryptografia asymetryczna?

Odpowiedź:

Dwóch:

- publicznego,
- prywatnego.

Zadanie 3. Prawda czy fałsz?

RSA jest kryptografią asymetryczną.

Odpowiedź: Prawda.

Uzasadnienie:

RSA używa klucza publicznego i prywatnego.

Zadanie 4. Prawda czy fałsz?

Kryptografia symetryczna jest zwykle szybsza od asymetrycznej.

Odpowiedź: Prawda.

Uzasadnienie:

Operacje symetryczne są dużo mniej kosztowne obliczeniowo.

Zadanie 5. Prawda czy fałsz?

W kryptografii asymetrycznej klucz publiczny musi być tajny.

Odpowiedź: Fałsz.

Uzasadnienie:

Klucz publiczny można publikować.

Zadanie 6.

Jaki problem rozwiązuje kryptografia asymetryczna?

Odpowiedź:

Rozwiązuje problem bezpiecznej wymiany klucza.

Zadanie 7.

Dlaczego w praktyce używa się obu typów kryptografii?

Odpowiedź:

Ponieważ:

- asymetryczna bezpiecznie wymienia klucze,
- symetryczna szybko szyfruje dane.

Zadanie 8.

Podaj przykłady kryptografii asymetrycznej.

Odpowiedź:

- RSA,
- ElGamal,
- Diffie–Hellman.

Zadanie 9.

Do czego najlepiej nadaje się kryptografia symetryczna?

Odpowiedź:

Do szybkiego szyfrowania dużych ilości danych i plików.

3.8 Dlaczego RSA działa

Definicja

RSA działa dzięki własnościom arytmetyki modularnej oraz funkcji Eulera.

Najważniejsza idea:

po zaszyfrowaniu i odszyfrowaniu wiadomości otrzymujemy z powrotem tę samą wartość.

W RSA:

$$M \rightarrow C \rightarrow M$$

czyli:

$$(M^e)^d \equiv M \pmod{n}$$

Bezpieczeństwo RSA opiera się na trudności rozkładu liczby:

$$n = pq$$

na czynniki pierwsze.

Najważniejsze wzory

Szyfrowanie:

$$C = M^e \pmod{n}$$

Deszyfrowanie:

$$M = C^d \pmod{n}$$

Kluczowa własność:

$$ed \equiv 1 \pmod{\varphi(n)}$$

czyli istnieje liczba:

$$k$$

taka, że:

$$ed = 1 + k\varphi(n)$$

Twierdzenie Eulera:

$$M^{\varphi(n)} \equiv 1 \pmod{n}$$

gdy:

$$\text{NWD}(M, n) = 1$$

Dlatego:

$$M^{ed} = M^{1+k\varphi(n)}$$

$$= M \cdot (M^{\varphi(n)})^k$$

$$\equiv M \cdot 1^k$$

$$\equiv M \pmod{n}$$

Własności i zasady

- RSA działa dzięki funkcji Eulera.
- Kluczowe jest:

$$ed \equiv 1 \pmod{\varphi(n)}$$

- Po szyfrowaniu i odszyfrowaniu wracamy do:

$$M$$

- Wszystkie działania wykonujemy modulo:

$$n$$

- Bezpieczeństwo RSA NIE opiera się na trudności potęgowania.
- Bezpieczeństwo opiera się na trudności faktoryzacji:

$$n = pq$$

Wyjaśnienie krok po kroku

Dlaczego deszyfrowanie działa:

1. Szyfrujemy:

$$C = M^e \pmod{n}$$

2. Deszyfrujemy:

$$C^d = (M^e)^d$$

$$= M^{ed}$$

3. Ponieważ:

$$ed = 1 + k\varphi(n)$$

mamy:

$$M^{ed} = M^{1+k\varphi(n)}$$

4. Rozdzielamy:

$$M \cdot (M^{\varphi(n)})^k$$

5. Z twierdzenia Eulera:

$$M^{\varphi(n)} \equiv 1 \pmod{n}$$

6. Ostatecznie:

$$M \cdot 1^k \equiv M$$

7. Otrzymujemy oryginalną wiadomość.

Przykład z zajęć

Dane:

$$p = 3, q = 11$$

$$n = 33$$

$$\varphi(33) = 20$$

Wybieramy:

$$e = 3$$

Szukamy:

$$d$$

takiego, że:

$$3d \equiv 1 \pmod{20}$$

$$d = 7$$

ponieważ:

$$3 \cdot 7 = 21$$

$$21 \equiv 1 \pmod{20}$$

Szyfrujemy:

$$M = 4$$

$$C = 4^3 \pmod{33}$$

$$64 \pmod{33} = 31$$

Deszyfrujemy:

$$31^7 \pmod{33}$$

Po obliczeniach:

$$4$$

Odpowiedź:

RSA poprawnie odzyskał wiadomość.

Typowe pułapki

- Łatwo pomylić:

$$\varphi(n)$$

z:

$$n$$

- Warunek:

$$ed \equiv 1 \pmod{\varphi(n)}$$

jest kluczowy.

- Wszystkie działania wykonujemy modulo:

$$n$$

- RSA nie działa „magicznie” — wynika z twierdzenia Eulera.

- Bezpieczeństwo RSA NIE wynika z tajności algorytmu.
- Bezpieczeństwo daje trudność faktoryzacji dużych liczb.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Prawda czy fałsz?

RSA działa dzięki funkcji Eulera.

Odpowiedź: Prawda.

Uzasadnienie:

RSA wykorzystuje twierdzenie Eulera i arytmetykę modularną.

Zadanie 2.

Jaki warunek musi spełniać:

$$e$$

oraz:

$$d$$

?

Odpowiedź:

$$ed \equiv 1 \pmod{\varphi(n)}$$

Zadanie 3. Prawda czy fałsz?

Bezpieczeństwo RSA opiera się na trudności logarytmu dyskretnego.

Odpowiedź: Fałsz.

Uzasadnienie:

RSA opiera się na trudności faktoryzacji liczb.

Zadanie 4. Prawda czy fałsz?

Po odszyfrowaniu RSA odzyskujemy oryginalną wiadomość.

Odpowiedź: Prawda.

Uzasadnienie:

Dzięki:

$$M^{ed} \equiv M \pmod{n}$$

Zadanie 5.

Na czym opiera się bezpieczeństwo RSA?

Odpowiedź:

Na trudności rozkładu dużej liczby:

$$n = pq$$

na czynniki pierwsze.

Zadanie 6.

Jakie twierdzenie matematyczne wykorzystuje RSA?

Odpowiedź:

Twierdzenie Eulera.

Zadanie 7. Prawda czy fałsz?

W RSA wszystkie działania wykonujemy modulo:

$$n$$

Odpowiedź: Prawda.

3.9 Known Plaintext Attack (na przykładzie ElGamala)

Definicja

Known Plaintext Attack (KPA) to rodzaj ataku kryptograficznego, w którym atakujący zna:

- fragment tekstu jawnego,
- odpowiadający mu szyfrogram.

Na zajęciach omawialiśmy KPA głównie na przykładzie ElGamala. Pokazywało to, dlaczego:

- potrzebna jest losowość,
- nie wolno powtarzać parametrów,
- nie wolno używać tego samego:

k

wielokrotnie.

Najważniejsze wzory

ElGamal:

$$c_1 = g^k \mod p$$

$$c_2 = m \cdot y^k \mod p$$

Jeśli użyjemy tego samego:

k

dla dwóch wiadomości:

m_1, m_2

otrzymujemy:

$$c_{2,1} = m_1 \cdot y^k \mod p$$

$$c_{2,2} = m_2 \cdot y^k \mod p$$

Dzieląc oba równania:

$$\frac{c_{2,2}}{c_{2,1}} \equiv \frac{m_2}{m_1} \pmod{p}$$

Jeśli znamy:

$$m_1$$

możemy odzyskać:

$$m_2$$

Własności i zasady

- ElGamal wymaga losowego:

$$k$$

dla każdej wiadomości.

- Ponowne użycie:

$$k$$

jest bardzo niebezpieczne.

- Known Plaintext Attack wykorzystuje znajomość części wiadomości.
- Losowość jest kluczowa dla bezpieczeństwa ElGamala.
- Ten sam plaintext nie powinien dawać tego samego szyfrogramu.

Jak działa atak

Schemat ataku:

1. Atakujący zna:

$$m_1$$

oraz odpowiadający szyfrogram.

2. Dla drugiej wiadomości użyto tego samego:

$$k$$

3. W obu szyfrogramach pojawia się ten sam:

$$y^k$$

4. Można obliczyć zależność pomiędzy wiadomościami.
5. Znając:

można odzyskać:

$$m_1$$

$$m_2$$

Najważniejsza idea:

Nigdy nie wolno ponownie używać tego samego:

$$k$$

Przykład z zajęć

Założmy, że użyto tego samego:

$$k$$

dla dwóch wiadomości.

Pierwsza wiadomość:

$$m_1 = 5$$

Druga wiadomość:

$$m_2 = 10$$

Szyfrogramy:

$$c_{2,1} = 15$$

$$c_{2,2} = 30$$

Atakujący zna:

$$m_1 = 5$$

oraz:

$$c_{2,1} = 15$$

Liczymy wspólny czynnik:

$$y^k = \frac{15}{5} = 3$$

Teraz odzyskujemy:

$$m_2 = \frac{30}{3} = 10$$

Odpowiedź:

Atakujący odzyskał drugą wiadomość.

Typowe pułapki

- Nigdy nie wolno używać tego samego:

k

wiele razy.

- ElGamal wymaga losowości.
- Ten sam plaintext może mieć różne szyfrogramy.
- Łatwo pomylić:

known plaintext attack

z:

brute force

- Problem nie wynika z matematyki ElGamala, lecz z błędnego użycia.

Przykładowe zadania z rozwiązaniem**Zadanie 1. Prawda czy fałsz?**

W ElGamalu można bezpiecznie używać tego samego:

k

dla wielu wiadomości.

Odpowiedź: Fałsz.

Uzasadnienie:

Powtórzenie:

k

umożliwia known plaintext attack.

Zadanie 2.

Dlaczego ElGamal potrzebuje losowego:

k

?

Odpowiedź:

Aby ta sama wiadomość nie dawała tego samego szyfrogramu oraz aby uniknąć ataków kryptograficznych.

Zadanie 3. Prawda czy fałsz?

Known Plaintext Attack zakłada znajomość części tekstu jawnego.

Odpowiedź: Prawda.

Uzasadnienie:

Atakujący zna fragment wiadomości oraz odpowiadający szyfrogram.

Zadanie 4.

Co może się stać po ponownym użyciu:

$$k$$

w ElGamalu?

Odpowiedź:

Możliwe jest odzyskanie innych wiadomości lub zależności między nimi.

Zadanie 5. Prawda czy fałsz?

Ten sam plaintext zawsze daje identyczny szyfrogram w ElGamalu.

Odpowiedź: Fałsz.

Uzasadnienie:

Każde szyfrowanie używa losowego:

$$k$$

Zadanie 6.

Na czym opiera się bezpieczeństwo ElGamala?

Odpowiedź:

Na problemie logarytmu dyskretnego oraz poprawnym użyciu losowego:

$$k$$

Zadanie 7.

Czy Known Plaintext Attack oznacza złamanie matematyki ElGamala?

Odpowiedź:

Nie.

Problem wynika z błędnego użycia algorytmu, np. ponownego użycia:

$$k$$

4 OpenPGP i podpis cyfrowy

4.1 OpenPGP Studio

Definicja

OpenPGP Studio to program używany do szyfrowania, odszyfrowywania oraz podpisywania danych.

Na zajęciach używaliśmy go do:

- generowania kluczy RSA,
- wymiany kluczy publicznych,
- szyfrowania plików,
- odszyfrowywania wiadomości,
- verify podpisu cyfrowego.

OpenPGP wykorzystuje kryptografię asymetryczną.

Każdy użytkownik posiada:

- klucz publiczny,
- klucz prywatny.

Najważniejsze informacje

Klucz publiczny:

- można udostępniać,
- służy do szyfrowania,
- służy do verify podpisu.

Klucz prywatny:

- musi być tajny,
- służy do odszyfrowywania,
- służy do podpisu cyfrowego.

Schemat z zajęć:

tekst → szyfrowanie → plik .pgp

oraz:

verify → sprawdzenie podpisu

Własności i zasady

- Każdy użytkownik generuje własną parę kluczy.
- Klucz publiczny można wysyłać innym osobom.
- Szyfrowanie wykonujemy kluczem publicznym odbiorcy.
- Odszyfrowanie wykonuje właściciel klucza prywatnego.
- Podpis cyfrowy tworzony jest kluczem prywatnym.
- Verify używa klucza publicznego.

Jak używaliśmy OpenPGP Studio

Generowanie kluczy:

1. Tworzymy parę kluczy RSA.
2. Otrzymujemy:
 - klucz publiczny,
 - klucz prywatny.

Szyfrowanie pliku:

1. Pobieramy klucz publiczny kolegi.
2. Wczytujemy plik tekstowy.
3. Szyfrujemy plik jego kluczem publicznym.
4. Powstaje zaszyfrowany plik:

.pgp

Odszyfrowanie:

1. Odbiorca używa własnego klucza prywatnego.
2. Odczytuje oryginalną wiadomość.

Verify podpisu:

1. Program sprawdza podpis cyfrowy.
2. Używany jest klucz publiczny nadawcy.
3. Można sprawdzić:
 - autora,
 - integralność danych.

Przykład z zajęć

Na zajęciach:

1. Wygenerowaliśmy klucze RSA.
2. Wysłaliśmy sobie klucze publiczne.
3. Kolega szyfrował plik naszym kluczem publicznym.
4. Otrzymywaliśmy zaszyfrowany plik:

.pgp

5. Odszyfrowywaliśmy go własnym kluczem prywatnym.
6. Używaliśmy:

Verify

do sprawdzania podpisu cyfrowego.

Najważniejsza obserwacja:

tylko właściciel klucza prywatnego mógł odszyfrować wiadomość.

Typowe pułapki

- Nie wolno udostępniać klucza prywatnego.
- Łatwo pomylić:
 - szyfrowanie,
 - podpis cyfrowy.
- Verify NIE odszyfrowuje danych.
- Verify sprawdza:
 - podpis,
 - integralność,
 - autora.
- Klucz publiczny służy do szyfrowania i verify.
- Klucz prywatny służy do odszyfrowania i podpisu.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Którego klucza używa się do szyfrowania wiadomości w OpenPGP?

Odpowiedź:

Klucza publicznego odbiorcy.

Zadanie 2.

Którego klucza używa się do odszyfrowania wiadomości?

Odpowiedź:

Klucza prywatnego odbiorcy.

Zadanie 3. Prawda czy fałsz?

Klucz prywatny można bezpiecznie wysyłać innym użytkownikom.

Odpowiedź: Fałsz.

Uzasadnienie:

Klucz prywatny musi pozostać tajny.

Zadanie 4. Prawda czy fałsz?

Verify służy do sprawdzania podpisu cyfrowego.

Odpowiedź: Prawda.

Uzasadnienie:

Verify sprawdza autora i integralność danych.

Zadanie 5.

Co sprawdza verify?

Odpowiedź:

- poprawność podpisu,
- autora wiadomości,
- integralność danych.

Zadanie 6. Prawda czy fałsz?

Tylko właściciel klucza prywatnego może odszyfrować wiadomość.

Odpowiedź: Prawda.

Zadanie 7.

Jakie operacje wykonywaliśmy w OpenPGP Studio?

Odpowiedź:

- generowanie kluczy,
- szyfrowanie,
- odszyfrowywanie,
- podpis cyfrowy,
- verify podpisu.

4.2 Podpis cyfrowy / Verify

Definicja

Podpis cyfrowy służy do potwierdzenia:

- autora wiadomości,
- integralności danych,
- autentyczności pliku.

Podpis cyfrowy NIE służy głównie do ukrywania treści wiadomości.

Na zajęciach używaliśmy podpisu cyfrowego w OpenPGP Studio przy użyciu funkcji:

Verify

Najważniejsze informacje

Podpis cyfrowy tworzy się:

kluczem prywatnym

Verify podpisu wykonuje się:

kluczem publicznym

Schemat:

plik → hash → podpis

Podpis cyfrowy zwykle podpisuje hash pliku, a nie cały plik.

Własności i zasady

- Podpis cyfrowy potwierdza autora wiadomości.
- Verify sprawdza poprawność podpisu.
- Zmiana choć jednego znaku powoduje błędny podpis.
- Podpis wykorzystuje kryptografię asymetryczną.
- Klucz prywatny tworzy podpis.
- Klucz publiczny wykonuje verify.
- Podpis cyfrowy zapewnia integralność danych.

Jak działa podpis cyfrowy

Tworzenie podpisu:

1. Tworzymy hash wiadomości.
2. Hash podpisujemy kluczem prywatnym.
3. Powstaje podpis cyfrowy.

Verify podpisu:

1. Odbiorca tworzy hash otrzymanego pliku.
2. Odbiorca używa klucza publicznego nadawcy.
3. Program porównuje hashe.
4. Jeśli są identyczne:
 - podpis jest poprawny,
 - dane nie zostały zmienione.

Przykład z zajęć

W OpenPGP Studio:

1. Tworzyliśmy podpis cyfrowy pliku.
2. Program używał klucza prywatnego.
3. Odbiorca używał:

Verify

4. Verify sprawdzał podpis przy użyciu klucza publicznego.

Jeśli plik został zmodyfikowany:

Verify = błąd

Jeśli plik był oryginalny:

Verify = OK

Typowe pułapki

- Podpis cyfrowy NIE szyfruje wiadomości.

- Verify NIE odszyfrowuje danych.
- Łatwo pomylić:
 - podpis cyfrowy,
 - szyfrowanie.
- Nawet mała zmiana pliku unieważnia podpis.
- Klucz prywatny musi pozostać tajny.
- Verify używa klucza publicznego.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Którego klucza używa się do tworzenia podpisu cyfrowego?

Odpowiedź:

Klucza prywatnego.

Zadanie 2.

Którego klucza używa się do verify podpisu?

Odpowiedź:

Klucza publicznego.

Zadanie 3. Prawda czy fałsz?

Podpis cyfrowy służy do szyfrowania wiadomości.

Odpowiedź:

Fałsz.

Uzasadnienie:

Podpis cyfrowy potwierdza autora i integralność danych.

Zadanie 4. Prawda czy fałsz?

Zmiana jednego znaku może unieważnić podpis cyfrowy.

Odpowiedź:

Prawda.

Uzasadnienie:

Zmienia się hash wiadomości.

Zadanie 5.

Co sprawdza verify?

Odpowiedź:

- poprawność podpisu,
- integralność danych,
- autora wiadomości.

Zadanie 6. Prawda czy fałsz?

Podpis cyfrowy wykorzystuje funkcje hashujące.

Odpowiedź: Prawda.

Uzasadnienie:

Podpisuje się hash wiadomości, a nie cały plik.

Zadanie 7.

Co się stanie, jeśli zmodyfikujemy podpisany plik?

Odpowiedź:

Verify wykryje zmianę i podpis będzie niepoprawny.

Zadanie 8. Prawda czy fałsz?

Verify używa klucza prywatnego.

Odpowiedź: Fałsz.

Uzasadnienie:

Verify używa klucza publicznego.

4.3 Integralność danych

Definicja

Integralność danych oznacza, że dane nie zostały zmienione podczas:

- przesyłania,
- zapisu,
- przechowywania.

Integralność odpowiada na pytanie:

„Czy dane są dokładnie takie same jak oryginał?”

Na zajęciach integralność pojawiała się przy:

- podpisie cyfrowym,
- verify,
- funkcjach hashujących,
- OpenPGP.

Najważniejsze informacje

Integralność często sprawdza się za pomocą:

- hashy,
- podpisu cyfrowego,
- verify.

Schemat:

plik $\rightarrow$ hash

Jeśli hash się zmieni:

dane zostały zmodyfikowane

Podpis cyfrowy:

hash $\rightarrow$ podpis

Verify sprawdza zgodność hashy.

Własności i zasady

- Integralność NIE oznacza poufności.
- Dane mogą być jawne, ale nadal integralne.
- Nawet zmiana jednego bitu zmienia hash.
- Funkcje hashujące służą do wykrywania zmian danych.
- Podpis cyfrowy zapewnia integralność oraz autentyczność.
- Verify sprawdza integralność podpisanych danych.

Jak sprawdzana jest integralność

Schemat działania:

1. Tworzymy hash pliku.
2. Wysyłamy plik.
3. Odbiorca oblicza hash ponownie.
4. Porównujemy oba hashe.
5. Jeśli hashe są identyczne:

dane nie zostały zmienione

6. Jeśli hashe są różne:

dane zostały zmodyfikowane

Przy podpisie cyfrowym:

1. Tworzony jest hash wiadomości.
2. Hash podpisuje się kluczem prywatnym.
3. Verify sprawdza zgodność hashy.

Przykład z zajęć

W OpenPGP Studio:

1. Podpisywaliśmy plik cyfrowo.
2. Program tworzył hash pliku.
3. Verify sprawdzał poprawność podpisu.

Jeśli plik został zmodyfikowany:

Verify = błąd

Jeśli plik był identyczny:

Verify = OK

Zmiana nawet jednego znaku powodowała zmianę hashy.

Typowe pułapki

- Integralność NIE oznacza szyfrowania.
- Hash NIE ukrywa danych.
- Łatwo pomylić:
 - integralność,
 - poufność.
- Verify sprawdza integralność, a nie odszyfrowuje danych.
- Nawet mała zmiana pliku zmienia hash.
- Podpis cyfrowy wykorzystuje funkcje hashujące.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Co oznacza integralność danych?

Odpowiedź:

Oznacza, że dane nie zostały zmienione.

Zadanie 2.

Jak najczęściej sprawdza się integralność danych?

Odpowiedź:

Za pomocą:

- hashy,
- podpisu cyfrowego,
- verify.

Zadanie 3. Prawda czy fałsz?

Integralność oznacza ukrywanie danych.

Odpowiedź: Fałsz.

Uzasadnienie:

Integralność oznacza wykrywanie zmian danych.

Zadanie 4. Prawda czy fałsz?

Zmiana jednego znaku może zmienić hash.

Odpowiedź: Prawda.

Uzasadnienie:

Funkcje hashujące są bardzo czułe na zmiany danych.

Zadanie 5.

Co sprawdza verify?

Odpowiedź:

- integralność danych,
- poprawność podpisu,
- autentyczność autora.

Zadanie 6. Prawda czy fałsz?

Hash może służyć do sprawdzania integralności danych.

Odpowiedź: Prawda.

Zadanie 7.

Co się stanie, jeśli dane zostaną zmienione po podpisaniu?

Odpowiedź:

Verify wykryje zmianę i podpis stanie się niepoprawny.

Zadanie 8. Prawda czy fałsz?

Integralność i poufność to to samo.

Odpowiedź: Fałsz.

Uzasadnienie:

Poufność ukrywa dane, a integralność wykrywa ich modyfikację.

5 Hashe i kolizije

5.1 Funkcje hashujące / funkcje skrótu

Definicja

Funkcja hashująca, czyli funkcja skrótu, zamienia dane wejściowe dowolnej długości na krótki wynik liczbowy lub tekstowy nazywany hashem.

Hash nie jest szyfrowaniem, ponieważ nie służy do odzyskiwania oryginalnej wiadomości.

Funkcje hashujące są używane do sprawdzania integralności danych, porównywania plików, podpisów cyfrowych, tablic hashujących oraz przechowywania haseł.

Najważniejsze informacje

Ogólny zapis funkcji hashującej:

$$H(x) = h$$

gdzie:

- x – dane wejściowe,
- H – funkcja hashująca,
- h – hash / skrót.

Przykład:

$$H(\text{tekst}) = 123456$$

Dobra funkcja hashująca powinna:

- działać szybko,
- dawać ten sam wynik dla tych samych danych,
- silnie zmieniać wynik po małej zmianie danych,
- minimalizować kolizje,
- być trudna do odwrócenia.

Własności i zasady

- Ta sama wiadomość zawsze daje ten sam hash.
- Mała zmiana danych zwykle powoduje dużą zmianę hashu.
- Hash ma zwykle stałą długość.
- Funkcja hashująca jest jednokierunkowa.
- Hash nie służy do szyfrowania wiadomości.

- Dwie różne wiadomości mogą mieć ten sam hash – to jest kolizja.

Metoda krok po kroku

Jak działa funkcja hashująca:

1. Bierzemy dane wejściowe, np. tekst lub plik.
2. Funkcja przetwarza dane według określonego algorytmu.
3. Wynikiem jest hash.
4. Dla tych samych danych hash powinien być taki sam.
5. Przy zmianie danych hash powinien się zmienić.

Schemat:

$\text{dane} \rightarrow H(\text{dane}) \rightarrow \text{hash}$

Przykład z zajęć

Dla tekstu:

test

funkcja DJB2 może zwrócić przykładowy hash:

2090756197

Dla tekstu z inną kolejnością liter:

tset

wynik będzie inny.

To pokazuje, że mała zmiana danych może zmienić wynik funkcji hashującej.

Typowe pułapki

- Hash to nie szyfr.
- Hasha nie odszyfrowuje się normalnie.
- Kolizje są możliwe.
- Funkcje DJB2 i Adler32 nie są bezpieczne kryptograficznie.
- Hash nie zapewnia poufności danych.
- Funkcja hashująca może służyć do integralności, ale sama nie potwierdza autora.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Czym jest funkcja hashująca?

Odpowiedź:

Funkcja hashująca zamienia dane wejściowe na skrót, czyli hash.

Zadanie 2. Prawda czy fałsz?

Hash jest szyfrowaniem.

Odpowiedź: Fałsz.

Uzasadnienie:

Hash jest funkcją jednokierunkową i nie służy do odzyskiwania oryginalnej wiadomości.

Zadanie 3. Prawda czy fałsz?

Ta sama wiadomość zawsze powinna dawać ten sam hash.

Odpowiedź: Prawda.

Zadanie 4. Co to jest kolizja?

Odpowiedź:

Kolizja występuje wtedy, gdy dwie różne wiadomości mają ten sam hash.

Zadanie 5. Do czego używa się funkcji hashujących?

Odpowiedź:

Funkcje hashujące są używane m.in. do:

- sprawdzania integralności danych,
- podpisów cyfrowych,
- tablic hashujących,
- przechowywania haseł,
- porównywania plików.

Zadanie 6. Prawda czy fałsz?

DJB2 i Adler32 są bezpieczne kryptograficznie.

Odpowiedź: Fałsz.

Uzasadnienie:

To proste funkcje hashujące/checksumy, ale nie są odporne kryptograficznie.

5.2 DJB / DJB2

Definicja

DJB2 to prosta funkcja hashująca stworzona przez Daniela J. Bernsteina. Przyjmuje tekst jako wejście i zwraca liczbę będącą haszem tego tekstu. Na zajęciach używaliśmy DJB2 do:

- obliczania hashy tekstów,
- testowania kolizji,
- budowania tablic hashów,
- łańcuchów hash $\rightarrow$ redukcja $\rightarrow$ hash.

DJB2 jest szybki i prosty, ale nie jest bezpieczny kryptograficznie.

Najważniejsze wzory

Wartość początkowa:

$$H_0 = 5381$$

Dla każdego znaku:

$$H = H \cdot 33 + ASCII(znaku)$$

Równoważny zapis:

$$H = H \cdot 32 + H + ASCII(znaku)$$

Zapis bitowy:

$$H = ((H \ll 5) + H) + ASCII(znaku)$$

ponieważ:

$$H \ll 5 = H \cdot 32$$

W programie stosowaliśmy obcięcie do 32 bitów:

$$H = H \& 0xFFFFFFFF$$

Własności i zasady

- DJB2 działa znak po znaku.
- Każdy znak zamieniany jest na kod ASCII.
- Startujemy od:

$$5381$$

- W każdym kroku mnożymy dotychczasowy hash przez:

$$33$$

- Następnie dodajemy kod ASCII znaku.
- W programie wynik był ograniczany do 32 bitów.
- DJB2 nie nadaje się do haseł ani podpisów cyfrowych.

Metoda krok po kroku

Jak liczyć DJB2 ręcznie:

1. Ustaw wartość początkową:

$$H = 5381$$

2. Weź pierwszy znak tekstu.

3. Zamień znak na ASCII.

4. Oblicz:

$$H = H \cdot 33 + ASCII$$

5. Przejdź do kolejnego znaku.

6. Powtarzaj do końca tekstu.

7. Jeśli liczymy jak w programie, obcinamy wynik do 32 bitów.

Schemat:

$$tekst \rightarrow ASCII \rightarrow H \cdot 33 + ASCII \rightarrow hash$$

Przykład z zajęć

Oblicz hash DJB2 dla tekstu:

A

Kod ASCII:

$$A = 65$$

Start:

$$H_0 = 5381$$

Liczymy:

$$H = 5381 \cdot 33 + 65$$

$$5381 \cdot 33 = 177573$$

$$177573 + 65 = 177638$$

Odpowiedź:

$$DJB2(A) = 177638$$

Dla dłuższego tekstu wykonujemy ten sam krok dla każdego znaku.

Typowe pułapki

- Nie zapomnij o wartości początkowej:

$$5381$$

- Wzór używa:

$$33$$

a nie 32.

- Zapis:

$$(H \ll 5) + H$$

oznacza:

$$H \cdot 32 + H = H \cdot 33$$

- Trzeba używać kodów ASCII znaków.
- Małe i duże litery mają różne kody ASCII.
- DJB2 nie jest odporny kryptograficznie.

Przykładowe zadania z rozwiązaniem

Zadanie 1. Oblicz DJB2 dla znaku:

$$A$$

Rozwiązanie:

$$H_0 = 5381$$

$$A = 65$$

$$H = 5381 \cdot 33 + 65$$

$$H = 177638$$

Odpowiedź:

$$177638$$

Zadanie 2. Oblicz pierwszy krok DJB2 dla znaku:

$$a$$

Rozwiązanie:

$$a = 97$$

$$H = 5381 \cdot 33 + 97$$

$$H = 177670$$

Odpowiedź:

$$177670$$

Zadanie 3. Prawda czy fałsz?

DJB2 zaczyna od wartości:

$$H_0 = 5381$$

Odpowiedź: Prawda.

Zadanie 4. Prawda czy fałsz?

W DJB2 wykonujemy:

$$H = H \cdot 33 + ASCII$$

Odpowiedź: Prawda.

Zadanie 5. Prawda czy fałsz?

DJB2 jest bezpieczny kryptograficznie i nadaje się do haseł.

Odpowiedź: Fałsz.

Uzasadnienie:

DJB2 jest prostą funkcją hashującą, ale nie jest odporna kryptograficznie.

Zadanie 6. Co oznacza zapis $H \ll 5$?

Odpowiedź:

Przesunięcie bitowe w lewo o 5, czyli mnożenie przez:

$$2^5 = 32$$

Dlatego:

$$(H \ll 5) + H = 32H + H = 33H$$

Zadanie 7. Do czego używaliśmy DJB2 na zajęciach?

Odpowiedź:

Do:

- liczenia hashy tekstu,
- testowania kolizji,
- tworzenia tablic hashów,
- budowania łańcuchów hash-redukcja.

5.3 Adler32

Definicja

Adler32 to prosta funkcja checksum / hash używana do wykrywania błędów danych. Została zaprojektowana jako szybsza alternatywa dla CRC32.

Adler32 NIE jest funkcją bezpieczną kryptograficznie.

Na zajęciach używaliśmy jej do:

- liczenia hashy,
- sprawdzania integralności danych,
- porównania z DJB2,
- analizy kolizji.

Najważniejsze wzory

Adler32 używa dwóch sum:

$$A$$

oraz:

$$B$$

Wartości początkowe:

$$A = 1$$

$$B = 0$$

Dla każdego bajtu:

$$A = (A + data) \mod 65521$$

$$B = (B + A) \mod 65521$$

Końcowy hash:

$$hash = (B \ll 16) | A$$

gdzie:

- $\ll 16$ oznacza przesunięcie bitowe,
- $|$ oznacza OR bitowe.

Własności i zasady

- Adler32 działa na bajtach danych.
- Funkcja używa dwóch sum:

$$A, B$$

- Modulo:

$$65521$$

jest największą liczbą pierwszą mniejszą od:

$$2^{16}$$

- Adler32 jest szybki obliczeniowo.
- Nie nadaje się do kryptografii.
- Drobną zmianę danych zmienia checksumę.

Metoda krok po kroku

Jak liczyć Adler32 ręcznie:

1. Ustaw:

$$A = 1$$

$$B = 0$$

2. Weź pierwszy znak.
3. Zamień go na ASCII.
4. Dodaj znak do:

$$A$$

5. Dodaj nowe:

$$A$$

do:

$$B$$

6. Powtarzaj dla wszystkich znaków.

7. Po każdym kroku wykonuj modulo:

$$65521$$

8. Na końcu połącz:

$$B$$

oraz:

$$A$$

Schemat:

$$tekst \rightarrow ASCII \rightarrow A, B \rightarrow hash$$

Przykład z zajęć

Oblicz Adler32 dla:

$$A$$

Kod ASCII:

$$65$$

Start:

$$A = 1$$

$$B = 0$$

Nowe:

$$A = (1 + 65) \mod 65521 = 66$$

Nowe:

$$B = (0 + 66) \mod 65521 = 66$$

Końcowy hash:

$$(B \ll 16) | A$$

$$(66 \ll 16) | 66$$

4325442

Odpowiedź:

$$\text{Adler32}(A) = 4325442$$

Typowe pułapki

- Adler32 NIE jest szyfrem.
- Trzeba pamiętać o dwóch zmiennych:

A, B

- Łatwo zapomnieć o:

$\text{mod } 65521$

- Operacja:

$<< 16$

to przesunięcie bitowe.

- Adler32 nie nadaje się do przechowywania haseł.
- Jest szybki, ale ma słabszą odporność na kolizje niż nowoczesne hashe.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Jakie są wartości początkowe w Adler32?

Odpowiedź:

$$A = 1$$

$$B = 0$$

Zadanie 2.

Jakie modulo wykorzystuje Adler32?

Odpowiedź:

65521

Zadanie 3. Prawda czy fałsz?

Adler32 używa dwóch sum kontrolnych.

Odpowiedź: Prawda.

Zadanie 4.

Oblicz nowe:

$$A$$

dla znaku:

$$A$$

Zakładamy:

$$A = 1$$

ASCII:

$$65$$

Rozwiązanie:

$$A = 1 + 65 = 66$$

Odpowiedź:

$$66$$

Zadanie 5. Prawda czy fałsz?

Adler32 jest bezpieczną funkcją kryptograficzną.

Odpowiedź: Fałsz.

Uzasadnienie:

To szybka checksuma, a nie kryptograficzna funkcja hashująca.

Zadanie 6.

Do czego używa się Adler32?

Odpowiedź:

Do:

- wykrywania błędów danych,
- integralności,
- checksum,
- szybkiego porównywania danych.

Zadanie 7. Co oznacza:

$$B \ll 16$$

Odpowiedź:

Przesunięcie bitowe liczby:

$$B$$

o 16 bitów w lewo.

5.4 Kolizje hashy

Definicja

Kolizja hashy występuje wtedy, gdy dwie różne wiadomości dają ten sam hash. Formalnie:

$$x_1 \neq x_2$$

ale:

$$H(x_1) = H(x_2)$$

Kolizje są możliwe dla każdej funkcji hashującej, ponieważ liczba możliwych wejść jest większa niż liczba możliwych hashy.

Na zajęciach badaliśmy kolizje m.in. dla:

- DJB2,
- Adler32,
- tablic hashów,
- rainbow tables.

Najważniejsze informacje

Kolizja:

$$H(A) = H(B)$$

mimo że:

$$A \neq B$$

Dobra funkcja hashująca powinna:

- minimalizować kolizje,
- rozprowadzać dane równomiernie,
- utrudniać znalezienie dwóch takich samych hashy.

Kolizje są związane z:

- birthday paradox,
- ograniczoną liczbą możliwych hashy,
- długością hashy.

Własności i zasady

- Każda funkcja hashująca może mieć kolizje.
- Im krótszy hash, tym większa szansa kolizji.
- Kolizja nie oznacza błędu programu.
- Kolizje są szczególnie ważne w kryptografii.
- Funkcje kryptograficzne powinny być odporne na:
 - przypadkowe kolizje,
 - celowe wyszukiwanie kolizji.
- DJB2 i Adler32 mają stosunkowo łatwe kolizje.

Jak szuka się kolizji

Schemat z zajęć:

1. Generujemy wiele losowych tekstów.
2. Dla każdego tekstu liczymy hash.
3. Zapisujemy hash w tablicy.
4. Sprawdzamy:

czy hash już istnieje
5. Jeśli tak:

znaleziono kolizję

Schemat:

$tekst_1 \rightarrow hash$

$tekst_2 \rightarrow hash$

$hash_1 = hash_2$

$\Rightarrow$ kolizja

Przykład z zajęć

Program generował losowe teksty:

ABCD

X7Q2

TEST

Następnie obliczał DJB2 lub Adler32.

Jeśli dwa różne teksty dawały ten sam hash:

$$H(ABCD) = H(X7Q2)$$

oznaczało to kolizję.

Przy małej liczbie testów kolizji mogło nie być.

Przy większej liczbie danych kolizje pojawiały się częściej.

Typowe pułapki

- Kolizje są zawsze możliwe.
- Brak kolizji w testach NIE oznacza, że funkcja jest idealna.
- Łatwo pomylić:
 - hash,
 - unikalny identyfikator.
- Im więcej danych, tym większa szansa kolizji.
- Krótkie hashe mają dużo większą liczbę kolizji.
- DJB2 i Adler32 nie są odporne kryptograficznie.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Co to jest kolizja hashy?

Odpowiedź:

Sytuacja, gdy różne dane mają ten sam hash.

Zadanie 2. Prawda czy fałsz?

Dwie różne wiadomości mogą mieć identyczny hash.

Odpowiedź: Prawda.

Zadanie 3. Prawda czy fałsz?

Kolizja oznacza błąd funkcji hashującej.

Odpowiedź: Fałsz.

Uzasadnienie:

Kolizje są matematycznie nieuniknione.

Zadanie 4.

Od czego zależy szansa kolizji?

Odpowiedź:

Od:

- długości hasha,
- liczby danych,
- jakości funkcji hashującej.

Zadanie 5. Prawda czy fałsz?

Im krótszy hash, tym większa szansa kolizji.

Odpowiedź: Prawda.

Zadanie 6.

Jak wykrywaliśmy kolizje na zajęciach?

Odpowiedź:

- generowaliśmy losowe teksty,
- liczyliśmy hashe,
- sprawdzaliśmy powtarzające się wartości.

Zadanie 7. Prawda czy fałsz?

Brak kolizji w małym teście oznacza, że funkcja jest bezkolizyjna.

Odpowiedź: Fałsz.

Uzasadnienie:

Kolizje mogą pojawić się przy większej liczbie danych.

Zadanie 8.

Dlaczego kolizje są problemem w kryptografii?

Odpowiedź:

Ponieważ mogą umożliwić podszywanie się pod dane lub osłabienie podpisów cyfrowych.

5.5 Birthday Paradox

Definicja

Birthday Paradox (paradoks urodzin) pokazuje, że kolizje pojawiają się dużo szybciej, niż intuicyjnie się wydaje.

W kryptografii oznacza to, że znalezienie dwóch takich samych hashy wymaga znacznie mniej prób niż sprawdzenie wszystkich możliwych hashy.

Na zajęciach łączyliśmy birthday paradox z:

- kolizjami hashy,
- rainbow tables,
- tablicami hashów,
- bezpieczeństwem funkcji hashujących.

Najważniejsze wzory

Dla funkcji hashującej o:

$$N$$

możliwych hashach:

szansa kolizji gwałtownie rośnie około:

$$\sqrt{N}$$

prób.

Przybliżony wzór:

$$P \approx 1 - e^{-\frac{k(k-1)}{2N}}$$

gdzie:

- P – prawdopodobieństwo kolizji,
- k – liczba prób,
- N – liczba możliwych hashy.

Dla 50% szansy kolizji:

$$k \approx 1.2\sqrt{N}$$

Własności i zasady

- Birthday paradox dotyczy prawdopodobieństwa kolizji.
- Kolizje pojawiają się szybciej niż intuicyjnie oczekujemy.

- Nie trzeba sprawdzać wszystkich hashy.
- Dla n -bitowego hasha bezpieczeństwo kolizji wynosi około:

$$2^{n/2}$$

- Im krótszy hash, tym łatwiejsze kolizje.
- Paradoks jest podstawą birthday attack.

Jak działa birthday paradox

Klasyczny przykład:

1. W grupie ludzi sprawdzamy daty urodzin.
2. Nie pytamy:

„Czy ktoś ma moje urodziny?”

lecz:

„Czy jakiegokolwiek dwie osoby mają te same urodziny?”

3. To drastycznie zwiększa szansę trafienia.

W kryptografii:

1. Generujemy wiele hashy.
2. Szukamy dowolnej pary:

$$H(x_1) = H(x_2)$$

3. Kolizja pojawia się około:

$$\sqrt{N}$$

prób.

Przykład z zajęć

Założmy funkcję hashującą dającą tylko:

10000

możliwych hashy.

Intuicyjnie mogłoby się wydawać, że potrzeba:

$$10000$$

prób do kolizji.

Birthday paradox pokazuje jednak, że kolizje zaczną pojawiać się już około:

$$\sqrt{10000} = 100$$

prób.

Dlatego przy większej liczbie testów DJB2 zaczynaliśmy obserwować kolizje.

Typowe pułapki

- Birthday paradox NIE mówi o konkretnym hashu.
- Szukamy dowolnej pary kolizyjnej.
- Łatwo pomylić:
 - „trafienie konkretnego hashu”,
 - „dowolną kolizję”.
- Bezpieczeństwo hashy jest około dwa razy mniejsze niż liczba bitów sugeruje.
- Dla 128-bitowego hasha odporność na kolizje wynosi około:

$$2^{64}$$

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Czego dotyczy birthday paradox?

Odpowiedź:

Prawdopodobieństwa wystąpienia kolizji.

Zadanie 2. Prawda czy fałsz?

Do znalezienia kolizji trzeba sprawdzić wszystkie możliwe hashe.

Odpowiedź: Fałsz.

Uzasadnienie:

Birthday paradox pokazuje, że kolizje pojawiają się dużo wcześniej.

Zadanie 3.

Po ilu próbach pojawiają się kolizje dla:

$$N = 10000$$

?

Rozwiązanie:

$$\sqrt{10000} = 100$$

Odpowiedź:

Około:

100

prób.

Zadanie 4. Prawda czy fałsz?

Birthday paradox jest związany z funkcjami hashującymi.

Odpowiedź: Prawda.

Zadanie 5.

Dlaczego birthday paradox jest problemem dla hashy?

Odpowiedź:

Ponieważ kolizje pojawiają się szybciej niż intuicyjnie oczekujemy.

Zadanie 6. Prawda czy fałsz?

Dla 128-bitowego hasha odporność na kolizje wynosi około:

$$2^{64}$$

Odpowiedź: Prawda.

Zadanie 7.

Co oznacza:

$$\sqrt{N}$$

w birthday paradox?

Odpowiedź:

Przybliżoną liczbę prób potrzebnych do pojawienia się kolizji.

5.6 Szansa kolizji i wzór birthday paradox

Definicja

Szansa kolizji określa prawdopodobieństwo, że dwie różne wiadomości będą miały ten sam hash.

W kryptografii problem ten opisuje birthday paradox.

Najważniejszy wniosek:

kolizje pojawiają się dużo szybciej niż intuicyjnie oczekujemy.

Na zajęciach łączyliśmy to z:

- funkcjami hashującymi,
- DJB2,
- Adler32,
- rainbow tables,
- bezpieczeństwem hashy.

Najważniejsze wzory

Przybliżone prawdopodobieństwo kolizji:

$$P \approx 1 - e^{-\frac{k(k-1)}{2N}}$$

gdzie:

- P – prawdopodobieństwo kolizji,
- k – liczba hashy,
- N – liczba możliwych hashy.

Przybliżenie dla 50% szansy kolizji:

$$k \approx 1.2\sqrt{N}$$

Dla hashy n -bitowych:

$$N = 2^n$$

Odporność na kolizje:

$$2^{n/2}$$

Własności i zasady

- Im więcej hashy generujemy, tym większa szansa kolizji.
- Krótsze hashe mają większą szansę kolizji.

- Kolizje pojawiają się około:

$$\sqrt{N}$$

prób.

- Nie trzeba sprawdzać wszystkich hashy.
- Dla hashy kryptograficznych długość hasha ma ogromne znaczenie.
- Birthday attack wykorzystuje właśnie birthday paradox.

Jak obliczać szansę kolizji

Schemat:

1. Określ liczbę możliwych hashy:

$$N$$

2. Określ liczbę wygenerowanych hashy:

$$k$$

3. Użyj wzoru:

$$P \approx 1 - e^{-\frac{k(k-1)}{2N}}$$

4. Dla szybkiego oszacowania użyj:

$$\sqrt{N}$$

5. Jeśli:

$$k \approx 1.2\sqrt{N}$$

to szansa kolizji wynosi około:

$$50\%$$

Przykład z zajęć

Założmy funkcję hashującą mającą:

$$N = 10000$$

możliwych hashy.

Liczymy:

$$\sqrt{10000} = 100$$

To oznacza, że już około:

$$100$$

losowych hashy daje dużą szansę kolizji.

Dla hashy 128-bitowych:

$$N = 2^{128}$$

Odporność na kolizje wynosi około:

$$2^{64}$$

Typowe pułapki

- Birthday paradox dotyczy dowolnej kolizji, a nie konkretnego hasha.
- Łatwo pomylić:
 - pełną przestrzeń hashy,
 - odporność na kolizje.
- Dla hashy n -bitowych bezpieczeństwo kolizji wynosi:

$$2^{n/2}$$

a nie:

$$2^n$$

- Krótkie hashe szybko prowadzą do kolizji.
- Brak kolizji w małym teście niczego nie gwarantuje.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Od czego zależy szansa kolizji?

Odpowiedź:

Od:

- liczby możliwych hashy,
- liczby wygenerowanych hashy,
- długości hashy.

Zadanie 2.

Po ilu próbach pojawi się około 50% szansy kolizji dla:

$$N = 10000$$

?

Rozwiązanie:

$$\sqrt{10000} = 100$$

$$1.2 \cdot 100 = 120$$

Odpowiedź:

Około:

$$120$$

prób.

Zadanie 3. Prawda czy fałsz?

Kolizje pojawiają się dopiero po sprawdzeniu wszystkich hashy.

Odpowiedź: Fałsz.

Uzasadnienie:

Birthday paradox pokazuje, że pojawiają się dużo wcześniej.

Zadanie 4.

Jaką odporność na kolizje ma hash 128-bitowy?

Rozwiązanie:

$$2^{128/2} = 2^{64}$$

Odpowiedź:

$$2^{64}$$

Zadanie 5. Prawda czy fałsz?

Im krótszy hash, tym większa szansa kolizji.

Odpowiedź: Prawda.

Zadanie 6.

Co oznacza:

$$\sqrt{N}$$

w birthday paradox?

Odpowiedź:

Przybliżoną liczbę prób potrzebnych do pojawienia się kolizji.

Zadanie 7. Prawda czy fałsz?

Birthday attack wykorzystuje birthday paradox.

Odpowiedź: Prawda.

5.7 Rainbow Tables

Definicja

Rainbow Tables to specjalne tablice używane do odzyskiwania danych z hashy, najczęściej haseł.

Idea polega na wcześniejszym przygotowaniu ogromnej liczby:

$$\text{hash} \rightarrow \text{redukcja} \rightarrow \text{hash}$$

zamiast liczenia wszystkiego podczas ataku.

Rainbow tables są kompromisem między:

- czasem,
- pamięcią.

Na zajęciach omawialiśmy:

- łańcuchy hash,
- redukcję,
- tablice hashów,
- kolizje,
- odzyskiwanie tekstu z hasha.

Najważniejsze informacje

Schemat rainbow table:

$$\text{tekst} \rightarrow \text{hash} \rightarrow \text{redukcja} \rightarrow \text{hash} \rightarrow \text{redukcja}$$

Hash:

$$H(x)$$

zamienia tekst na skrót.

Redukcja:

$$R(h)$$

zamienia hash na nowy tekst.

Przechowujemy zwykle tylko:

- początek łańcucha,
- koniec łańcucha.

Własności i zasady

- Rainbow tables służą do odwracania hashy.
- Nie przechowujemy wszystkich hashy osobno.
- Używa się funkcji redukcji.
- Łańcuchy pozwalają oszczędzać pamięć.
- Im dłuższe łańcuchy, tym większe pokrycie przestrzeni.
- Kolizje mogą skracać skuteczność rainbow tables.
- Solenie haseł praktycznie niszczy skuteczność rainbow tables.

Jak działają rainbow tables

Tworzenie tablicy:

1. Wybieramy tekst początkowy.
2. Liczymy hash:

$$H(x)$$

3. Wykonujemy redukcję:

$$R(H(x))$$

4. Otrzymujemy nowy tekst.
5. Powtarzamy wiele razy.
6. Zapisujemy:

- początek łańcucha,
- koniec łańcucha.

Atak:

1. Otrzymujemy hash ofiary.
2. Próbujemy odtworzyć łańcuch:

$$hash \rightarrow redukcja \rightarrow hash$$

3. Jeśli trafimy na koniec istniejącego łańcucha:

możemy odzyskać tekst

Przykład z zajęć

Założmy:

$$\text{tekst} = ABCD$$

Liczymy hash:

$$H(ABCD) = 1234$$

Redukcja:

$$R(1234) = QWER$$

Ponownie hash:

$$H(QWER) = 5678$$

Tworzymy łańcuch:

$$ABCD \rightarrow 1234 \rightarrow QWER \rightarrow 5678$$

W tablicy możemy zapisać tylko:

$$ABCD \rightarrow 5678$$

Typowe pułapki

- Redukcja NIE jest odwrotnością hash.
- Hash nie jest odwracany bezpośrednio.
- Rainbow tables działają głównie przeciw niesolonym hashom.
- Łatwo pomylić:
 - hash,
 - redukcję.
- Kolizje mogą powodować łączenie łańcuchów.
- Długie hasła znacząco utrudniają rainbow tables.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Do czego służą rainbow tables?

Odpowiedź:

Do odzyskiwania danych z hashy, np. haseł.

Zadanie 2. Prawda czy fałsz?

Rainbow tables przechowują wszystkie możliwe hashe osobno.

Odpowiedź: Fałsz.

Uzasadnienie:

Używają łańcuchów hash i redukcji.

Zadanie 3.

Co robi funkcja redukcji?

Odpowiedź:

Zamienia hash na nowy tekst wejściowy.

Zadanie 4. Prawda czy fałsz?

Redukcja jest odwrotnością funkcji hashującej.

Odpowiedź: Fałsz.

Uzasadnienie:

Redukcja tylko tworzy nowy tekst z hasha.

Zadanie 5.

Jak wygląda schemat rainbow tables?

Odpowiedź:

tekst $\rightarrow$ hash $\rightarrow$ redukcja $\rightarrow$ hash

Zadanie 6. Prawda czy fałsz?

Solenie haseł utrudnia działanie rainbow tables.

Odpowiedź: Prawda.

Zadanie 7.

Dlaczego rainbow tables oszczędzają pamięć?

Odpowiedź:

Ponieważ przechowują głównie początki i końce łańcuchów.

Zadanie 8.

Jakie elementy były używane na zajęciach przy rainbow tables?

Odpowiedź:

- DJB2,
- redukcja,
- tablice hashów,
- kolizje,
- łańcuchy hash.

5.8 Redukcja

Definicja

Redukcja to funkcja używana w rainbow tables do zamiany hasha na nowy tekst wejściowy.

Redukcja NIE jest odszyfrowaniem hasha.

Jej zadaniem jest wygenerowanie nowego tekstu, który można ponownie zahashować.

Na zajęciach redukcja była używana w schemacie:

$$\text{tekst} \rightarrow \text{hash} \rightarrow \text{redukcja} \rightarrow \text{tekst}$$

Najważniejsze informacje

Funkcja hashująca:

$$H(x) = h$$

Funkcja redukcji:

$$R(h) = x'$$

Schemat rainbow tables:

$$x \rightarrow H(x) \rightarrow R(H(x)) \rightarrow x'$$

Najważniejsze:

$$R(h) \neq H^{-1}(h)$$

Redukcja nie odwraca hashy.

Własności i zasady

- Redukcja zamienia hash na nowy tekst.
- Redukcja NIE odzyskuje oryginalnej wiadomości.
- Wynik redukcji musi należeć do przestrzeni możliwych haseł.
- Rainbow tables używają naprzemiennie:

$$\text{hash} \rightarrow \text{redukcja}$$

- Różne funkcje redukcji mogą dawać różne wyniki.
- Redukcja jest kompromisem między pamięcią i czasem.

Jak działa redukcja

Schemat działania:

1. Liczymy hash tekstu:

$$H(x)$$

2. Otrzymujemy hash:

$$h$$

3. Funkcja redukcji:

$$R(h)$$

tworzy nowy tekst.

4. Nowy tekst ponownie hashujemy.

5. Proces powtarzamy:

$$hash \rightarrow redukcja \rightarrow hash$$

Cel:

Budowanie długich łańcuchów do rainbow tables.

Przykład z zajęć

Założmy:

$$H(ABCD) = 1234$$

Funkcja redukcji:

$$R(1234) = QWER$$

Następnie:

$$H(QWER) = 5678$$

Tworzymy łańcuch:

$$ABCD \rightarrow 1234 \rightarrow QWER \rightarrow 5678$$

Najważniejsze:

$$QWER$$

nie jest oryginalnym tekstem.
To tylko nowy tekst wygenerowany przez redukcję.

Typowe pułapki

- Redukcja NIE jest odwrotnością hasha.
- Redukcja nie odzyskuje oryginalnych danych.
- Łatwo pomylić:
 - redukcję,
 - odszyfrowanie.
- Wynik redukcji może być zupełnie innym tekstem.
- Kolizje mogą powodować łączenie łańcuchów.
- Redukcja działa razem z funkcją hashującą.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Do czego służy redukcja?

Odpowiedź:

Do zamiany hasha na nowy tekst wejściowy.

Zadanie 2. Prawda czy fałsz?

Redukcja odszyfrowuje hash.

Odpowiedź: Fałsz.

Uzasadnienie:

Redukcja tylko tworzy nowy tekst z hasha.

Zadanie 3.

Jak wygląda schemat rainbow tables?

Odpowiedź:

$$\text{tekst} \rightarrow \text{hash} \rightarrow \text{redukcja} \rightarrow \text{tekst}$$

Zadanie 4. Prawda czy fałsz?

Redukcja jest odwrotnością funkcji hashującej.

Odpowiedź: Fałsz.

Uzasadnienie:

Hash nie jest normalnie odwracalny.

Zadanie 5.

Co otrzymujemy po redukcji?

Odpowiedź:

Nowy tekst wejściowy do kolejnego hashowania.

Zadanie 6. Prawda czy fałsz?

Rainbow tables używają naprzemiennie funkcji hashującej i redukcji.

Odpowiedź: Prawda.

Zadanie 7.

Dlaczego redukcja jest potrzebna w rainbow tables?

Odpowiedź:

Pozwala budować długie łańcuchy:

$$hash \rightarrow redukcja \rightarrow hash$$

Zadanie 8.

Czy redukcja odzyskuje oryginalny tekst?

Odpowiedź:

Nie.

Tworzy tylko nowy tekst na podstawie hasha.

5.9 Tablice hashów

Definicja

Tablica hashów (hash table) to struktura danych umożliwiająca bardzo szybkie wyszukiwanie danych.

Działa poprzez:

- obliczenie hasha,
- przypisanie danych do odpowiedniego miejsca w tablicy.

Na zajęciach używaliśmy tablic hashów do:

- wykrywania kolizji,
- przechowywania hashy DJB2,
- testowania rainbow tables.

Najważniejsze informacje

Schemat działania:

$$\text{tekst} \rightarrow \text{hash} \rightarrow \text{indeks w tablicy}$$

Najczęściej:

$$\text{index} = H(x) \mod N$$

gdzie:

- $H(x)$ – hash,
- N – rozmiar tablicy.

Jeśli dwa elementy trafiają do tego samego indeksu:

kolizja

Własności i zasady

- Tablice hashów umożliwiają szybkie wyszukiwanie danych.
- Hash określa miejsce w tablicy.
- Kolizje są możliwe.
- Dobra funkcja hashująca równomiernie rozkłada dane.
- Rozmiar tablicy wpływa na liczbę kolizji.
- Zbyt mała tablica zwiększa liczbę kolizji.

Jak działa tablica hashów

Schemat działania:

1. Obliczamy hash:

$$H(x)$$

2. Liczymy indeks:

$$H(x) \bmod N$$

3. Wstawiamy dane do odpowiedniego miejsca tablicy.
4. Przy wyszukiwaniu ponownie liczymy hash.
5. Sprawdzamy odpowiedni indeks.

Kolizja:

Jeśli dwa elementy mają ten sam indeks:

$$index_1 = index_2$$

potrzebny jest mechanizm obsługi kolizji.

Przykład z zajęć

Założmy:

$$N = 10$$

Dla tekstu:

TEST

funkcja DJB2 daje:

123456

Liczymy indeks:

$$123456 \bmod 10 = 6$$

Tekst trafia do komórki:

tablica[6]

Drugi tekst może również trafić do:

co powoduje kolizję.

Typowe pułapki

- Hash i indeks to nie to samo.
- Kolizje są normalnym zjawiskiem.
- Mała tablica powoduje dużo kolizji.
- Zła funkcja hashująca nierówno rozkłada dane.
- Łatwo pomylić:
 - hash,
 - indeks tablicy.
- Tablica hashów nie gwarantuje braku kolizji.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Do czego służy tablica hashów?

Odpowiedź:

Do szybkiego przechowywania i wyszukiwania danych.

Zadanie 2.

Jak oblicza się indeks tablicy hashów?

Odpowiedź:

Najczęściej:

$$index = H(x) \mod N$$

Zadanie 3.

Oblicz indeks:

$$1234 \mod 10$$

Rozwiązanie:

$$1234 \mod 10 = 4$$

Odpowiedź:

$$4$$

Zadanie 4. Prawda czy fałsz?

Dwa różne teksty mogą trafić do tej samej komórki tablicy.

Odpowiedź: Prawda.

Uzasadnienie:

To jest kolizja.

Zadanie 5.

Co wpływa na liczbę kolizji?

Odpowiedź:

- rozmiar tablicy,
- jakość funkcji hashującej,
- liczba danych.

Zadanie 6. Prawda czy fałsz?

Hash i indeks tablicy to zawsze ta sama wartość.

Odpowiedź: Fałsz.

Uzasadnienie:

Indeks zwykle obliczamy jako:

$$H(x) \bmod N$$

Zadanie 7.

Co się stanie, jeśli tablica będzie zbyt mała?

Odpowiedź:

Wzrośnie liczba kolizji.

Zadanie 8.

Do czego używaliśmy tablic hashów na zajęciach?

Odpowiedź:

Do:

- wykrywania kolizji,
- przechowywania hashy,
- rainbow tables.

5.10 Łańcuchy hash $\rightarrow$ redukcja $\rightarrow$ hash

Definicja

Łańcuchy:

$$\text{hash} \rightarrow \text{redukcja} \rightarrow \text{hash}$$

są podstawą działania rainbow tables.

Idea polega na wielokrotnym wykonywaniu:

- funkcji hashującej,
- funkcji redukcji.

Pozwala to tworzyć długie łańcuchy reprezentujące ogromną liczbę możliwych haseł bez przechowywania wszystkich hashy osobno.

Na zajęciach używaliśmy tego schematu z:

- DJB2,
- tablicami hashów,
- rainbow tables,
- kolizjami.

Najważniejsze informacje

Schemat łańcucha:

$$x_0 \rightarrow H(x_0) \rightarrow R(H(x_0)) = x_1$$

$$x_1 \rightarrow H(x_1) \rightarrow R(H(x_1)) = x_2$$

Ogólny zapis:

$$x_i \rightarrow H(x_i) \rightarrow R(H(x_i)) = x_{i+1}$$

gdzie:

- H – funkcja hashująca,
- R – funkcja redukcji.

W rainbow tables zapisujemy zwykle:

- początek łańcucha,
- koniec łańcucha.

Własności i zasady

- Hash zamienia tekst na skrót.
- Redukcja zamienia hash na nowy tekst.
- Redukcja NIE odzyskuje oryginalnego tekstu.
- Łańcuchy pozwalają oszczędzać pamięć.
- Kolizje mogą powodować łączenie łańcuchów.
- Dłuższe łańcuchy pokrywają większą przestrzeń haseł.
- Rainbow tables są kompromisem:

pamięć $\leftrightarrow$ czas

Jak tworzy się łańcuch

Schemat działania:

1. Wybieramy tekst początkowy:

x_0

2. Liczymy hash:

$H(x_0)$

3. Wykonujemy redukcję:

$R(H(x_0))$

4. Otrzymujemy nowy tekst:

x_1

5. Powtarzamy proces wiele razy.

6. Zapisujemy:

- początek,
- koniec łańcucha.

Cel:

Zmniejszenie ilości pamięci potrzebnej do przechowywania ogromnej liczby hashy.

Przykład z zajęć

Startujemy od:

$ABCD$

Hash:

$$H(ABCD) = 1234$$

Redukcja:

$$R(1234) = QWER$$

Ponownie hash:

$$H(QWER) = 5678$$

Kolejna redukcja:

$$R(5678) = TEST$$

Łańcuch:

$$ABCD \rightarrow 1234 \rightarrow QWER \rightarrow 5678 \rightarrow TEST$$

W tablicy możemy przechować tylko:

$$ABCD \rightarrow TEST$$

Typowe pułapki

- Redukcja NIE jest odwrotnością hasha.
- Łatwo pomylić:
 - hash,
 - redukcję.
- Kolizje mogą łączyć różne łańcuchy.
- Wynik redukcji nie musi przypominać oryginalnego tekstu.
- Rainbow tables nie działają dobrze przeciw hashom z solą.
- Długie hasła utrudniają skuteczność rainbow tables.

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Jak wygląda schemat łańcucha rainbow tables?

Odpowiedź:

$$\text{tekst} \rightarrow \text{hash} \rightarrow \text{redukcja} \rightarrow \text{hash}$$

Zadanie 2.

Co robi funkcja redukcji?

Odpowiedź:

Zamienia hash na nowy tekst wejściowy.

Zadanie 3. Prawda czy fałsz?

Redukcja odzyskuje oryginalny tekst.

Odpowiedź: Fałsz.

Uzasadnienie:

Tworzy jedynie nowy tekst z hashu.

Zadanie 4.

Dlaczego używa się łańcuchów?

Odpowiedź:

Aby zmniejszyć ilość pamięci potrzebnej do przechowywania hashy.

Zadanie 5. Prawda czy fałsz?

Rainbow tables używają naprzemiennie hash i redukcji.

Odpowiedź: Prawda.

Zadanie 6.

Co zwykle zapisujemy w rainbow tables?

Odpowiedź:

- początek łańcucha,
- koniec łańcucha.

Zadanie 7. Prawda czy fałsz?

Kolizje mogą powodować łączenie łańcuchów.

Odpowiedź: Prawda.

Zadanie 8.

Dlaczego solenie utrudnia rainbow tables?

Odpowiedź:

Ponieważ ten sam tekst daje różne hashe dla różnych soli.

5.11 Operacje bitowe (<<, >>, AND, maski bitowe)

Definicja

Operacje bitowe działają bezpośrednio na bitach liczb binarnych.

Na zajęciach używaliśmy ich m.in. w:

- DJB2,
- Adler32,
- maskowaniu bitów,
- ograniczaniu hashy do 32 bitów.

Najważniejsze operacje:

- przesunięcie w lewo:

<<

- przesunięcie w prawo:

>>

- AND bitowy:

&

Najważniejsze informacje

Przesunięcie w lewo:

$$x \ll n$$

oznacza:

$$x \cdot 2^n$$

Przesunięcie w prawo:

$$x \gg n$$

oznacza:

$$\left\lfloor \frac{x}{2^n} \right\rfloor$$

AND bitowy:

$$a \& b$$

porównuje bity obu liczb.

Maska bitowa:

$$0xFFFFFFFF$$

pozwała zachować tylko 32 bity.

Przykład z DJB2:

$$(H \ll 5) + H$$

oznacza:

$$32H + H = 33H$$

Własności i zasady

- Operacje bitowe działają na reprezentacji binarnej.
- $$\ll$$
przesuwa bity w lewo.
- $$\gg$$
przesuwa bity w prawo.
- $$\&$$
służy do maskowania bitów.
- Maski bitowe pozwalają wyciągać lub usuwać fragmenty danych.
- Operacje bitowe są bardzo szybkie.

Jak działają operacje bitowe

Przesunięcie w lewo:

$$5 = 00000101$$
$$5 \ll 1$$
$$00001010$$

czyli:

10

Przesunięcie w prawo:

$$20 = 00010100$$

$$20 \gg 2$$

$$00000101$$

czyli:

$$5$$

AND bitowy:

$$13 = 1101$$

$$9 = 1001$$

$$1101 \& 1001 = 1001$$

czyli:

$$9$$

Przykład z zajęć

W DJB2 używaliśmy:

$$(H \ll 5) + H$$

Ponieważ:

$$H \ll 5 = H \cdot 32$$

otrzymujemy:

$$32H + H = 33H$$

W programie ograniczaliśmy hash do 32 bitów:

$$H \& 0xFFFFFFFF$$

Maska:

$0xFFFFFFFF$

zostawia tylko 32 najmłodsze bity.

Typowe pułapki

- $\ll$
nie oznacza zwykłego mnożenia tekstowego.
- $\gg$
może obcinać bity.
- Łatwo pomylić:
 - AND logiczny,
 - AND bitowy.
- Maski działają na bitach, nie na cyfrach dziesiętnych.
- $H \ll 5$
oznacza:
 $H \cdot 32$
a nie:
 $H \cdot 5$

Przykładowe zadania z rozwiązaniem

Zadanie 1.

Oblicz:

$$5 \ll 1$$

Rozwiązanie:

$$5 \cdot 2^1 = 10$$

Odpowiedź:

10

Zadanie 2.

Oblicz:

$$20 \gg 2$$

Rozwiązanie:

$$20/2^2 = 20/4 = 5$$

Odpowiedź:

$$5$$

Zadanie 3.

Co oznacza:

$$H \ll 5$$

Odpowiedź:

Przesunięcie bitowe w lewo o 5, czyli:

$$H \cdot 32$$

Zadanie 4. Prawda czy fałsz?

Operacja:

$$\&$$

to AND bitowy.

Odpowiedź: Prawda.

Zadanie 5.

Do czego służy maska:

$$0xFFFFFFFF$$

?

Odpowiedź:

Do pozostawienia tylko 32 bitów liczby.

Zadanie 6.

Oblicz:

$$13 \& 9$$

Rozwiązanie:

$$13 = 1101$$

$$9 = 1001$$

$$1101 \& 1001 = 1001$$

$$1001_2 = 9$$

Odpowiedź:

9

Zadanie 7. Prawda czy fałsz?

Operacje bitowe są używane w funkcjach hashujących.

Odpowiedź: Prawda.

Uzasadnienie:

Były używane m.in. w DJB2.

Zadanie 8.

Dlaczego operacje bitowe są popularne?

Odpowiedź:

Ponieważ są bardzo szybkie obliczeniowo.